

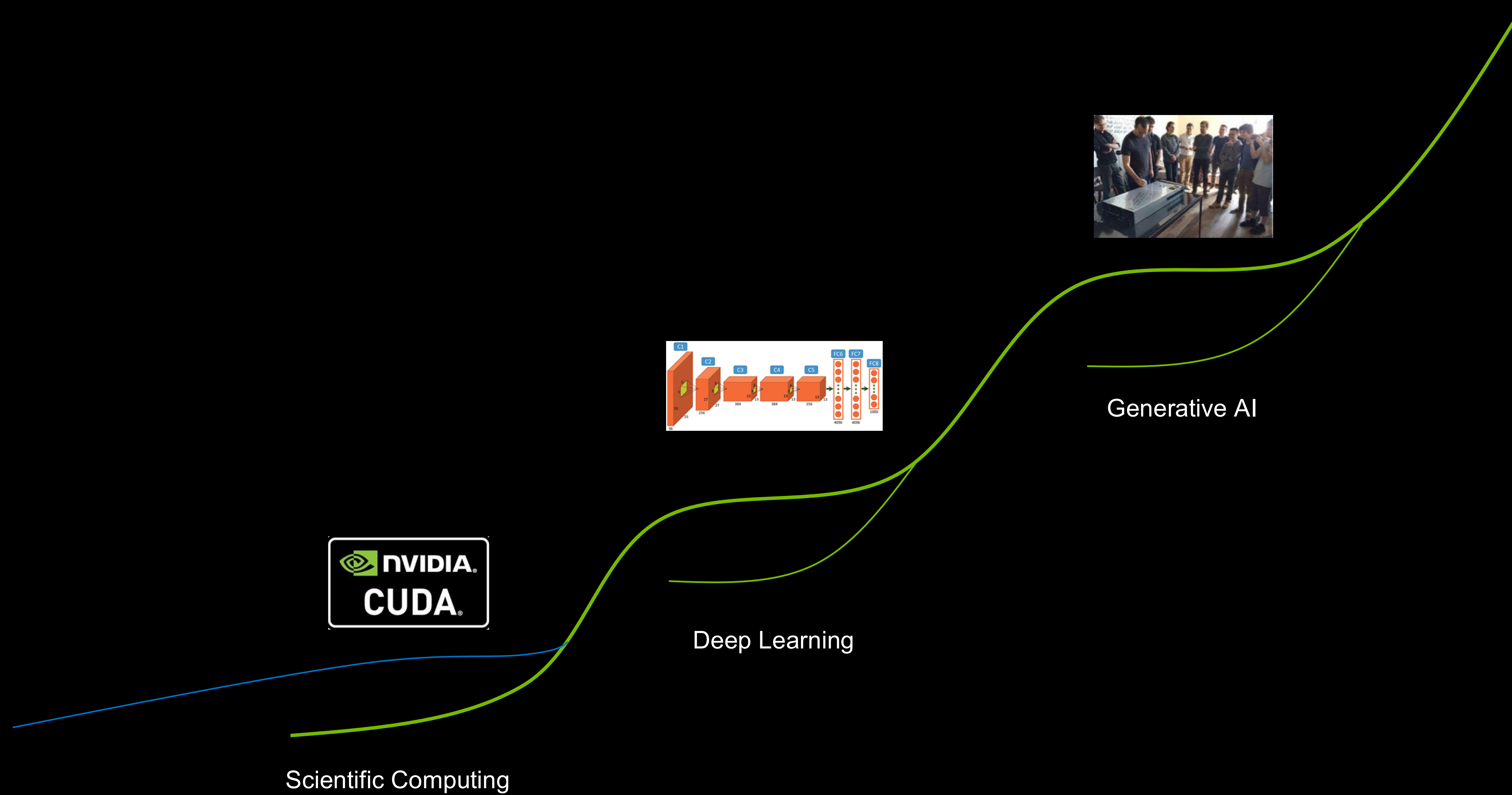


Building Quantum-GPU Supercomputing

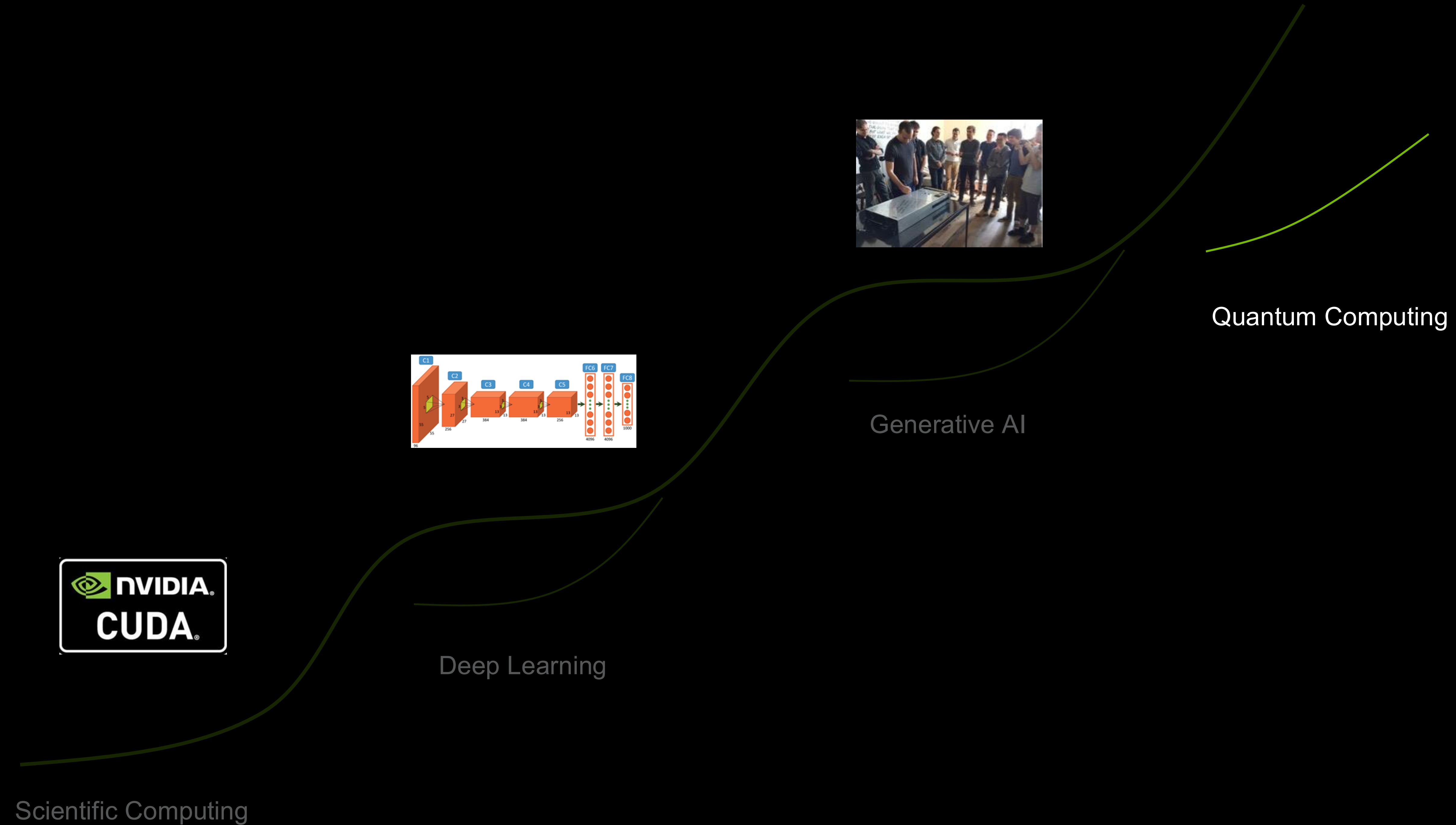
Pika Wang | Sr. Solutions Architect

SINICA × NCHC CUDA-Q Agentic Coding Bootcamp | Aug 7, 2026

NVIDIA's History of Enabling Computing Revolutions



NVIDIA's History of Enabling Computing Revolutions



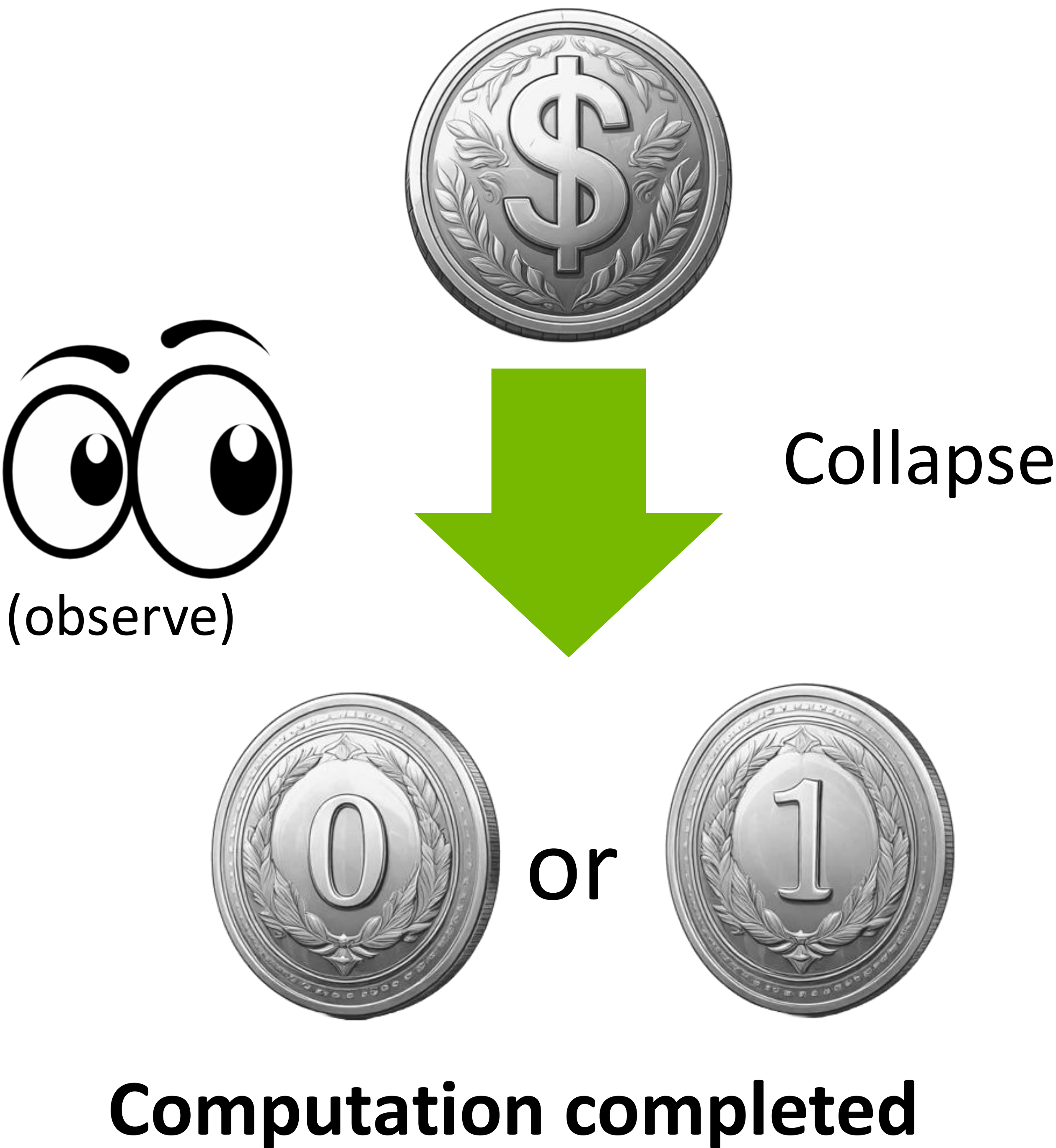
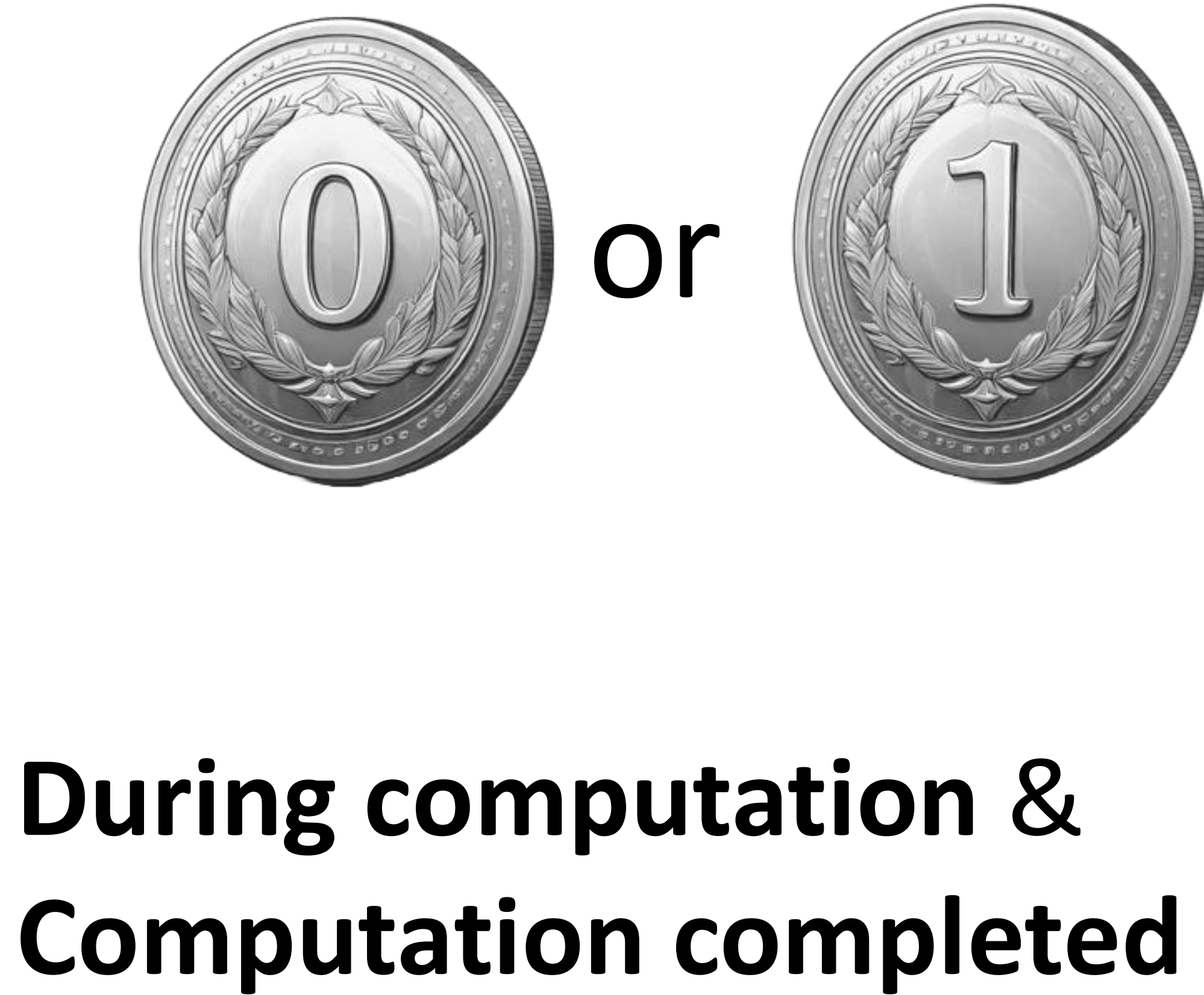
Qubits and Coins

A simplified yet informal illustration

Bit

Qubit

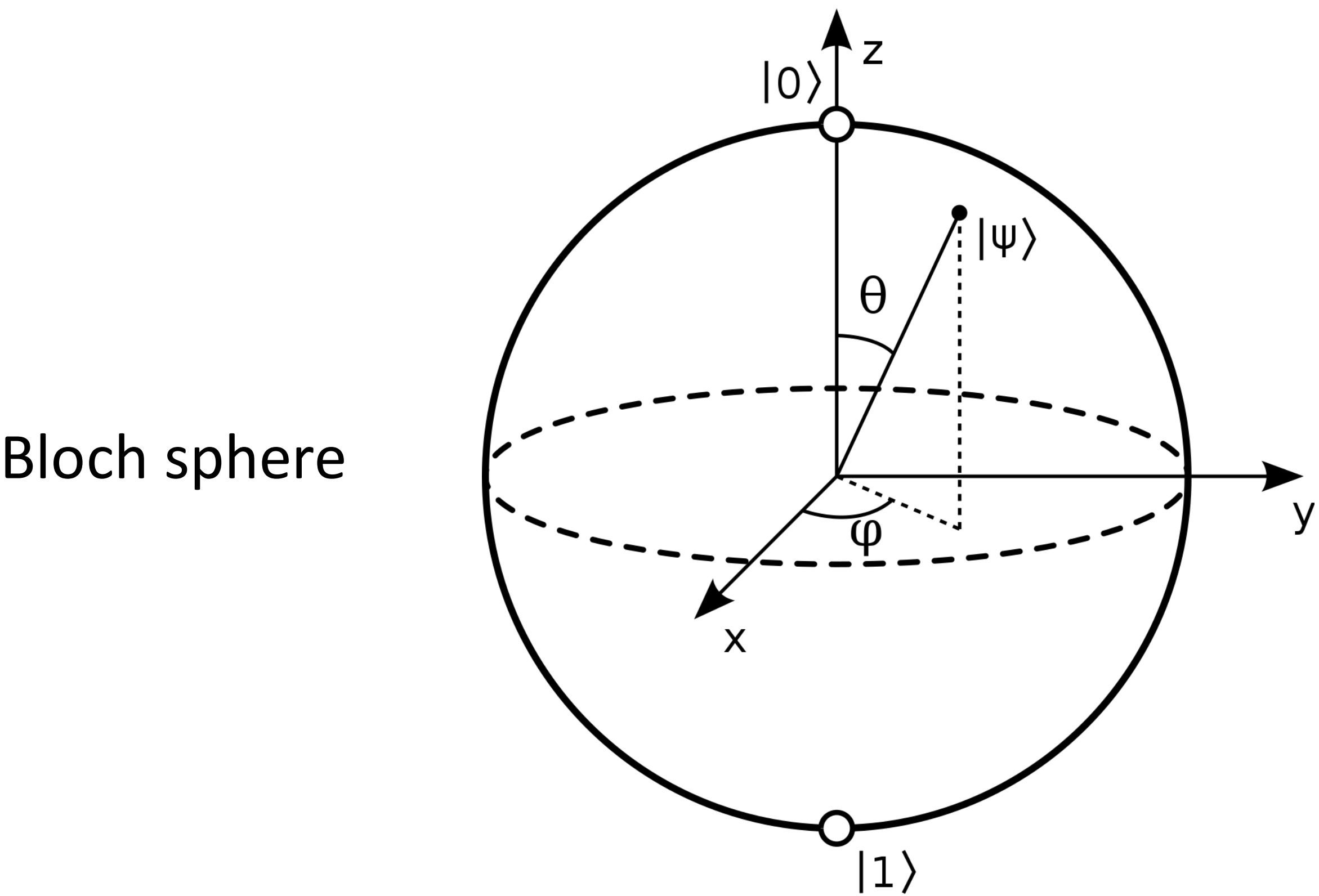
During computation



Qubits and Coins

Superposition – Let's dive into math

Single Qubit



Superposition of both 0 and 1

$$|\Psi\rangle = \cos(\theta/2)|0\rangle + e^{i\phi} \sin(\theta/2)|1\rangle$$

- Probability of measuring $|0\rangle$:

$$P(0) = |\cos(\theta/2)|^2 = \cos^2(\theta/2)$$

- Probability of measuring $|1\rangle$:

$$P(1) = |e^{i\phi} \sin(\theta/2)|^2 = \sin^2(\theta/2)$$

Since $\cos^2(\theta/2) + \sin^2(\theta/2) = 1$, this ensures a valid quantum probability distribution.

Multi Qubits

$$c_0|0\rangle + c_1|1\rangle = \begin{bmatrix} c_0 \\ c_1 \end{bmatrix}$$

$$c_{00}|00\rangle + c_{01}|01\rangle + c_{10}|10\rangle + c_{11}|11\rangle = \begin{bmatrix} c_{00} \\ c_{01} \\ c_{10} \\ c_{11} \end{bmatrix}$$

$\vdots$

$$c_{000}|000\rangle + c_{001}|001\rangle + c_{010}|010\rangle + c_{011}|011\rangle + c_{100}|100\rangle + c_{101}|101\rangle + c_{110}|110\rangle + c_{111}|111\rangle$$

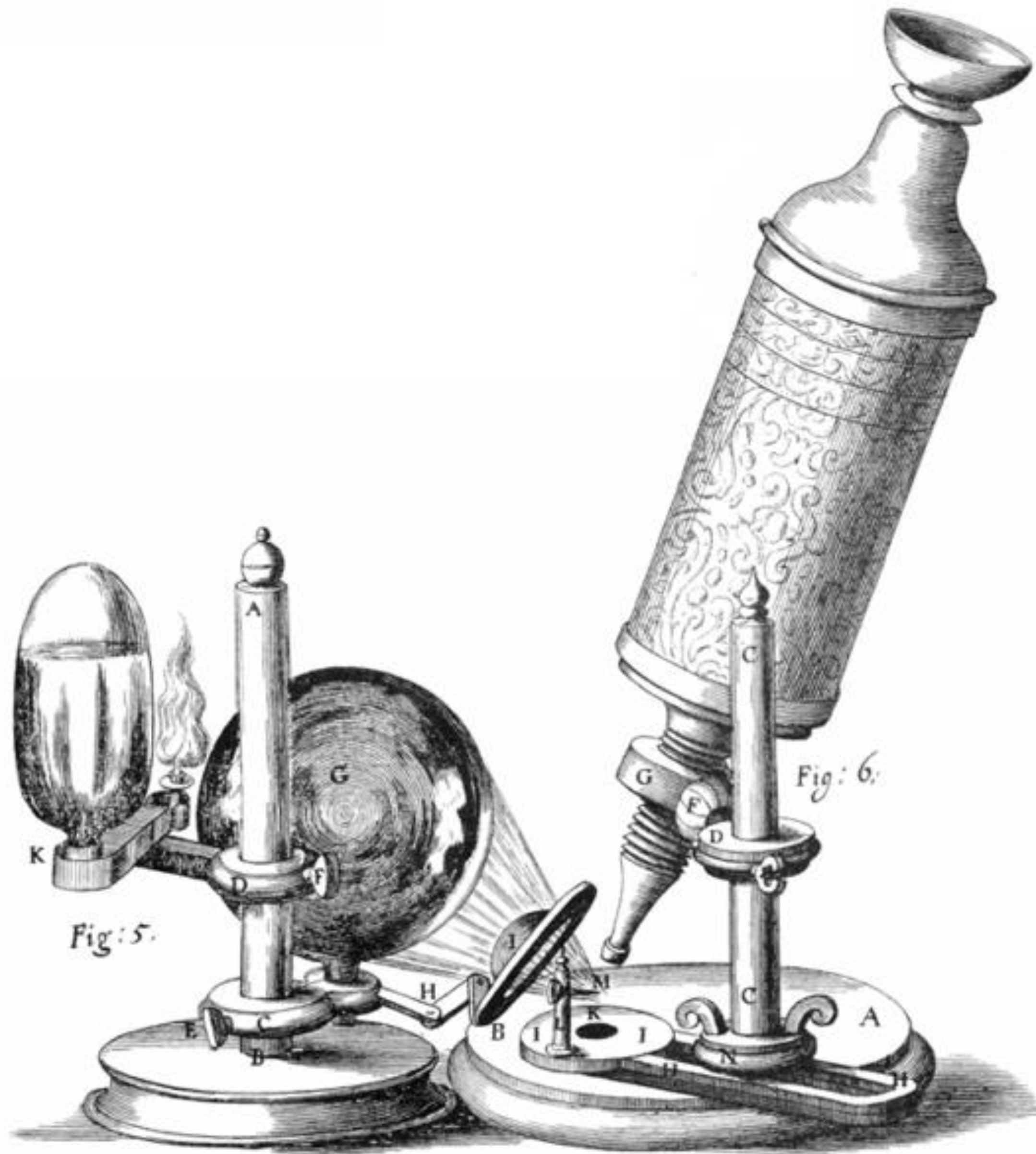
$$= \begin{bmatrix} c_{000} \\ c_{001} \\ c_{010} \\ c_{011} \\ c_{100} \\ c_{101} \\ c_{110} \\ c_{111} \end{bmatrix}$$

N qubits can represent
 2^N states simultaneously

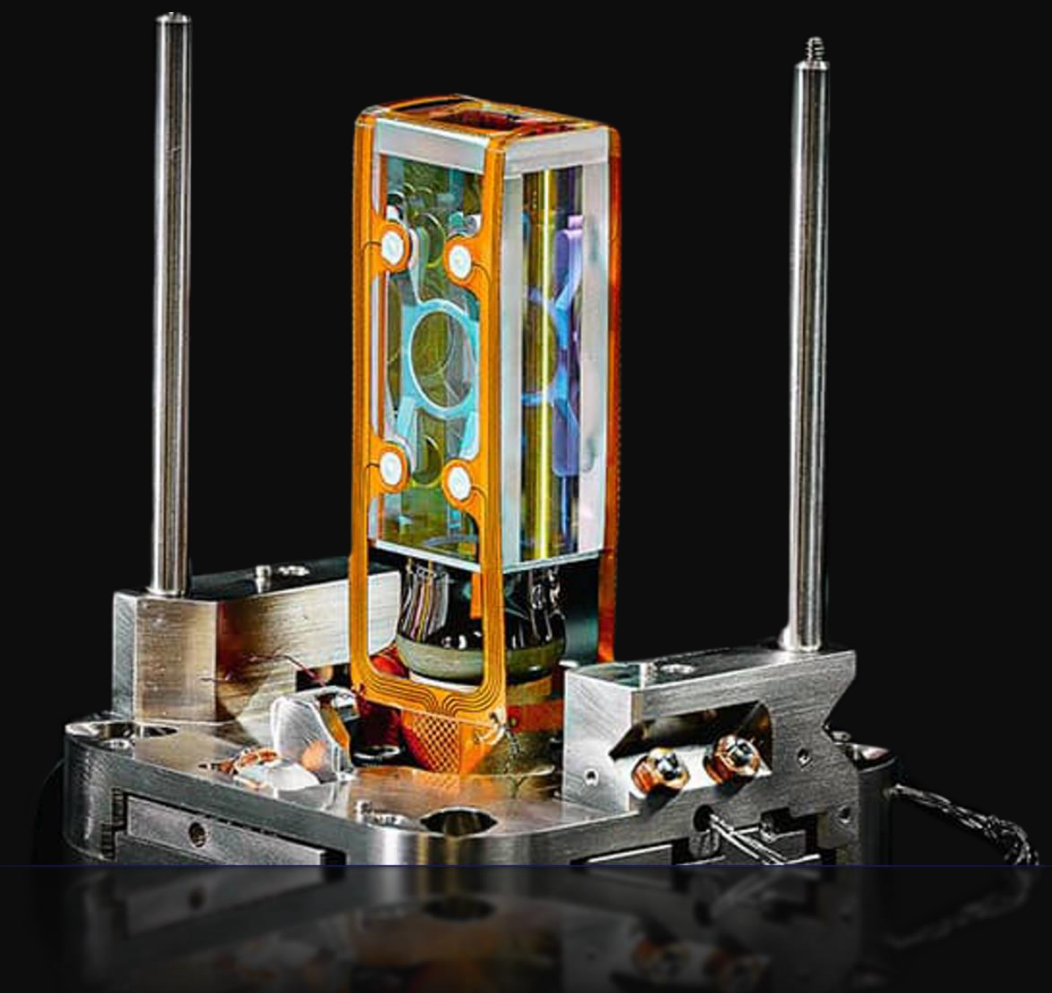
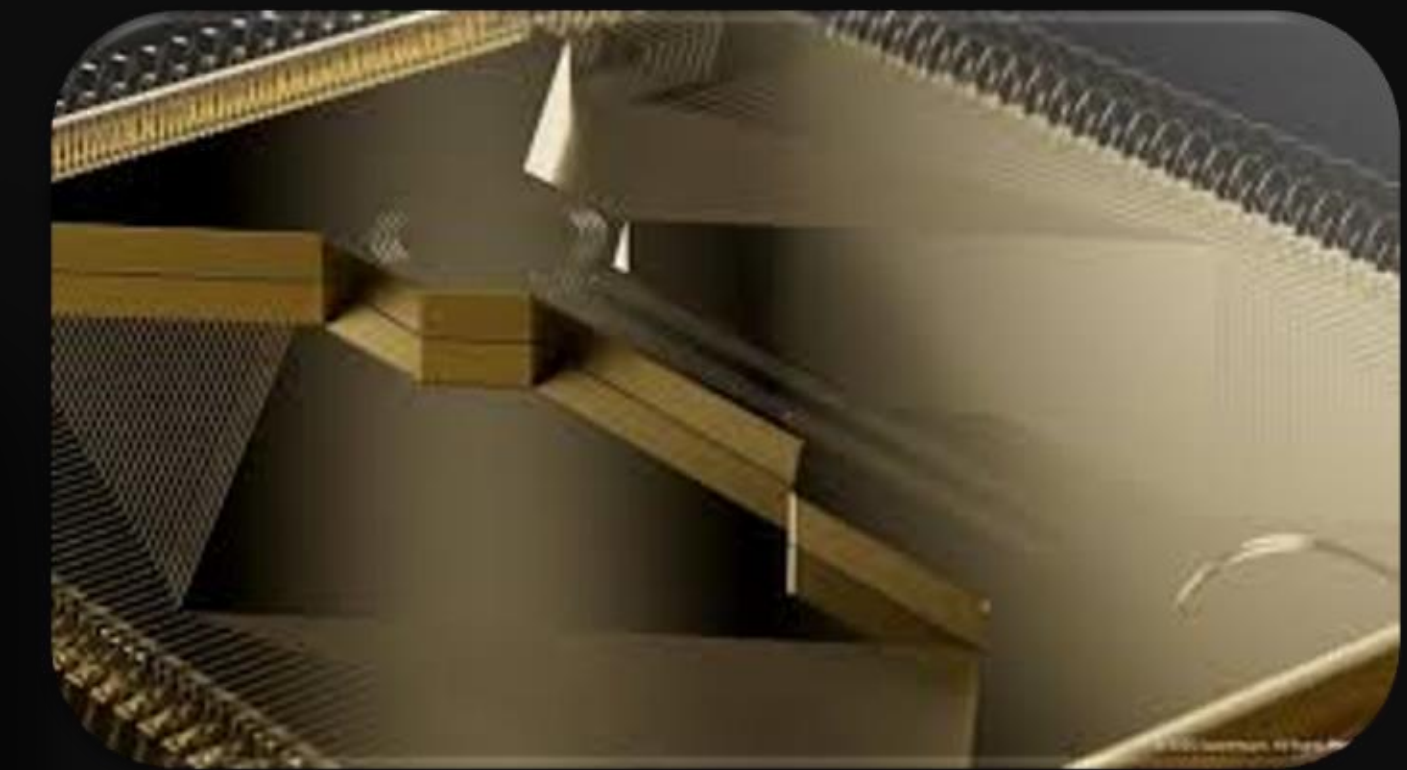
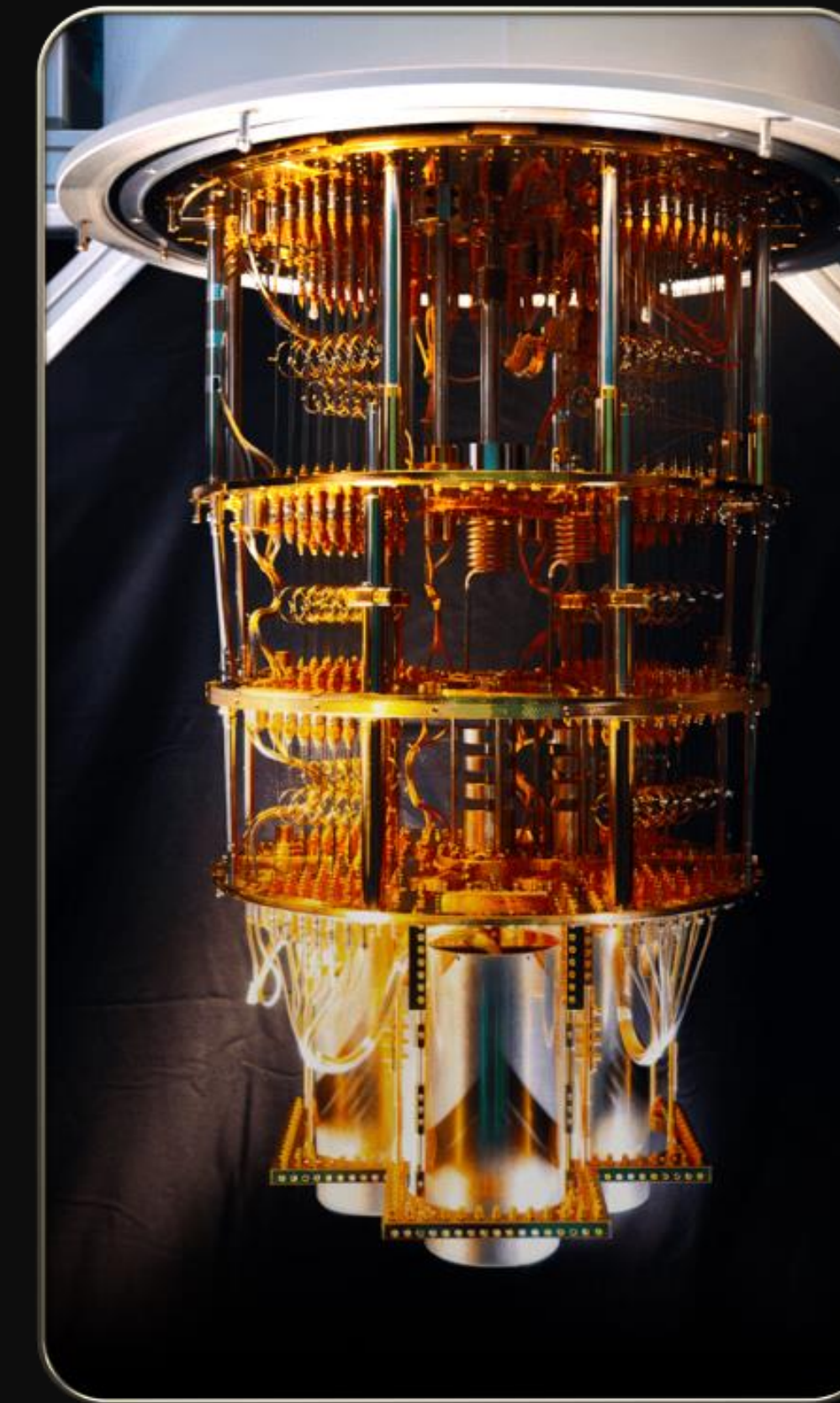


Qubit is more informative than classical bit!

The Next Great Instrument for Scientific Discovery



Robert Hooke's Microscope, 1664



Quantum Processors, today

AI and Quantum

AI to Advance Quantum Computing



Algorithms

QEC

Calibration

Digital Twins

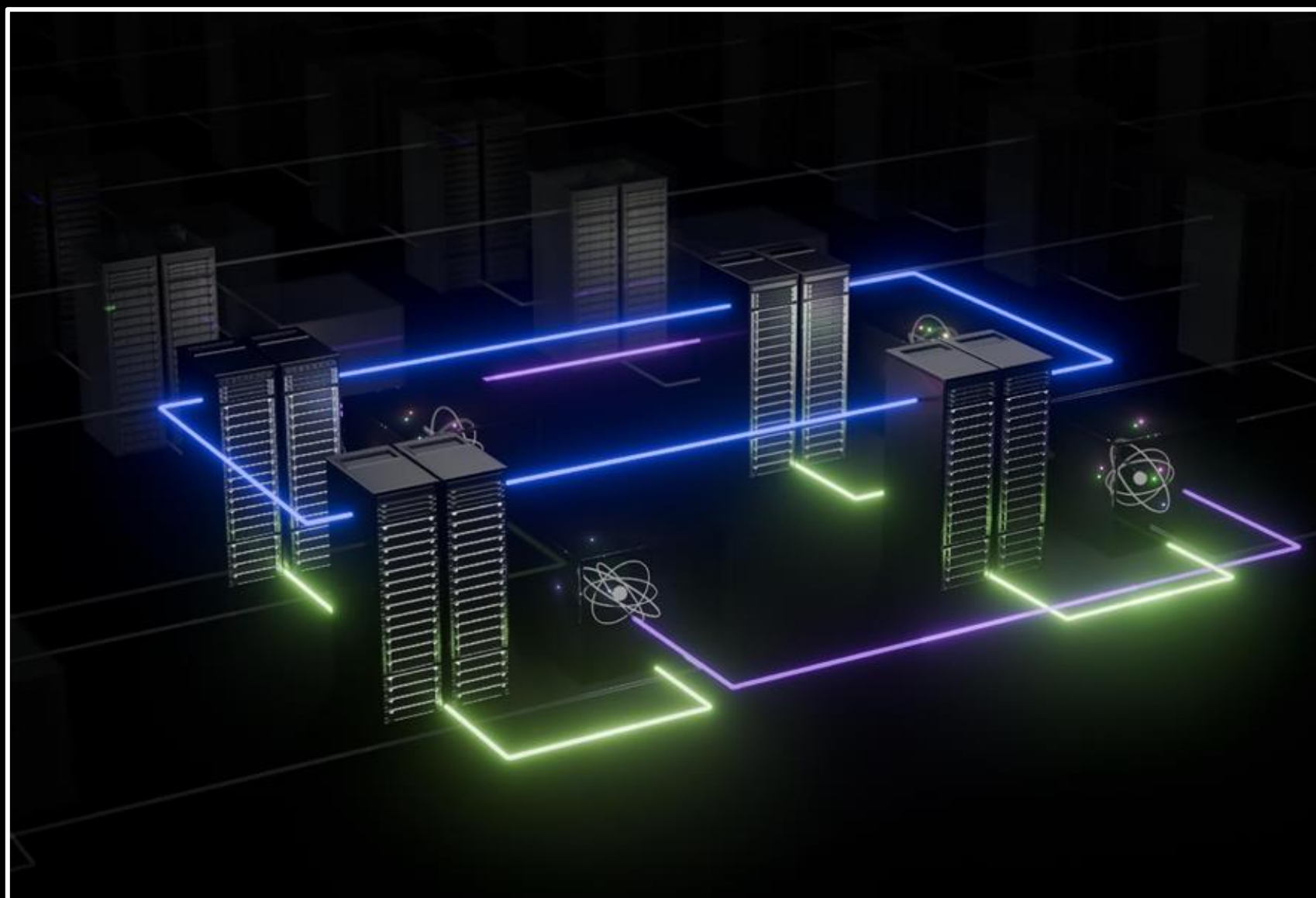
Quantum Computing to Advance AI



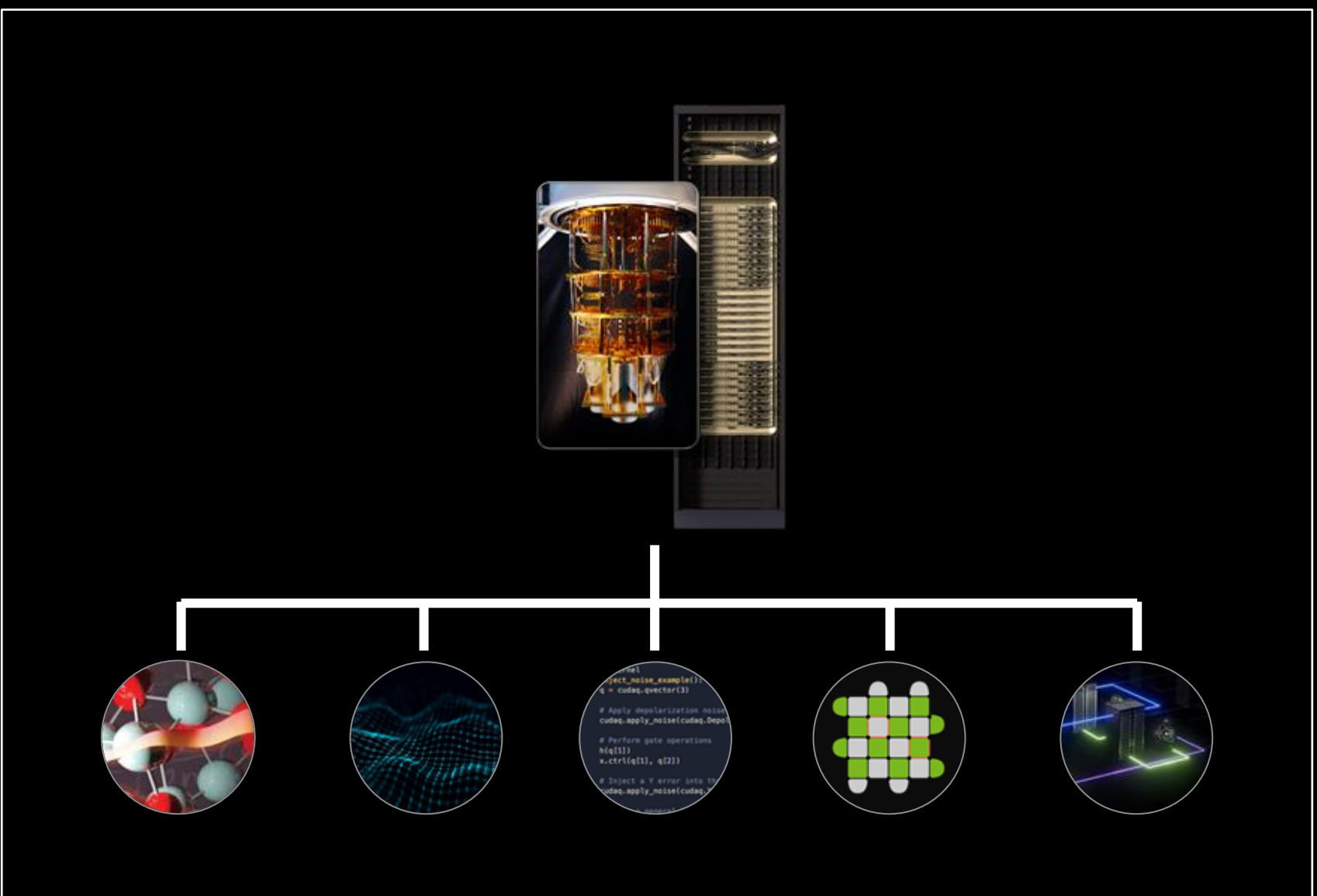
Hybrid Applications

Quantum Data Generation

Expand Frontier of
Accelerated Computing



Running useful quantum applications
and hardware requires **AI
supercomputing**



Developing useful quantum applications
and hardware requires
AI supercomputing

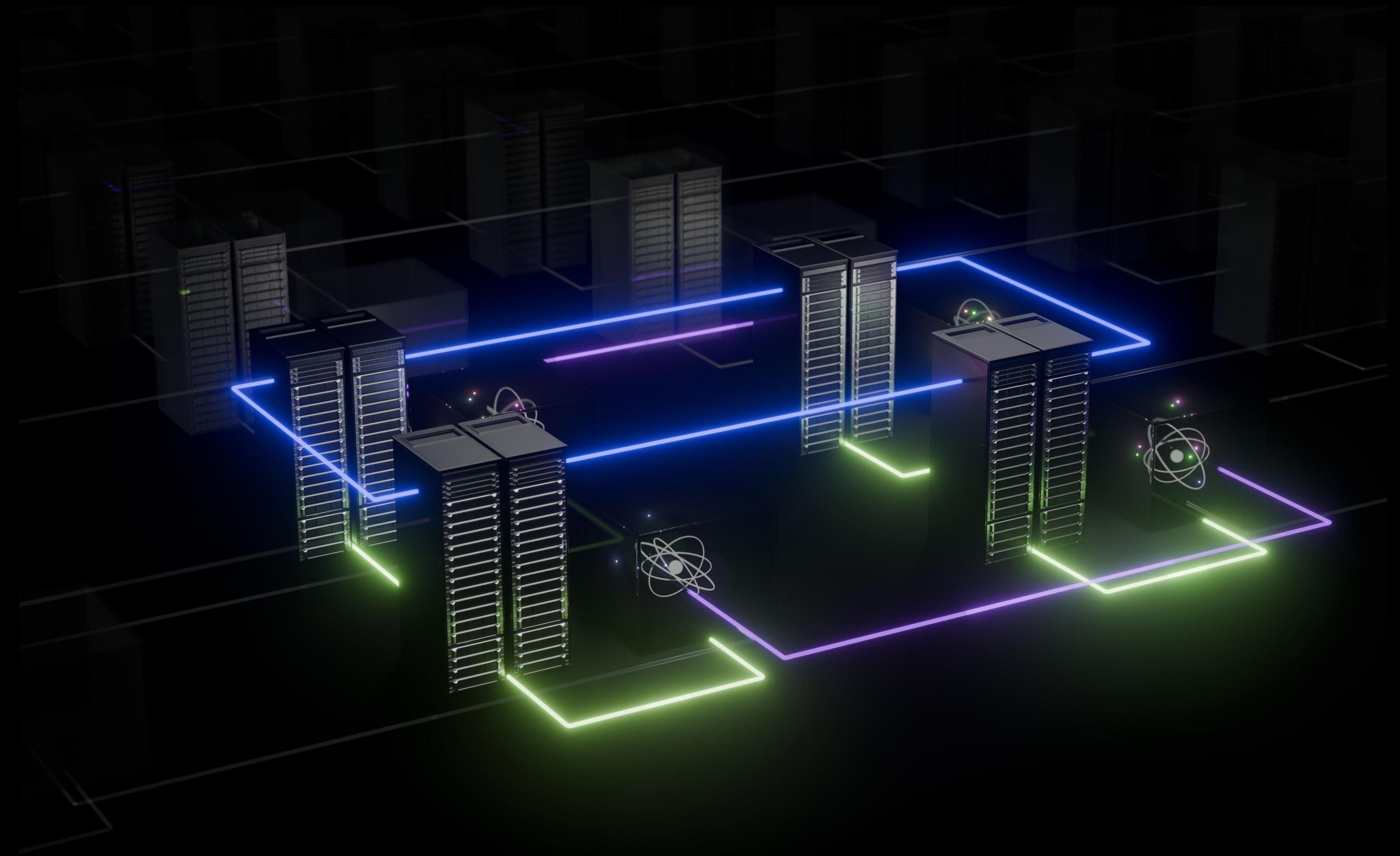
Quantum-GPU Supercomputing

All valuable quantum computing will be **hybrid**

GPUs and AI will prove essential

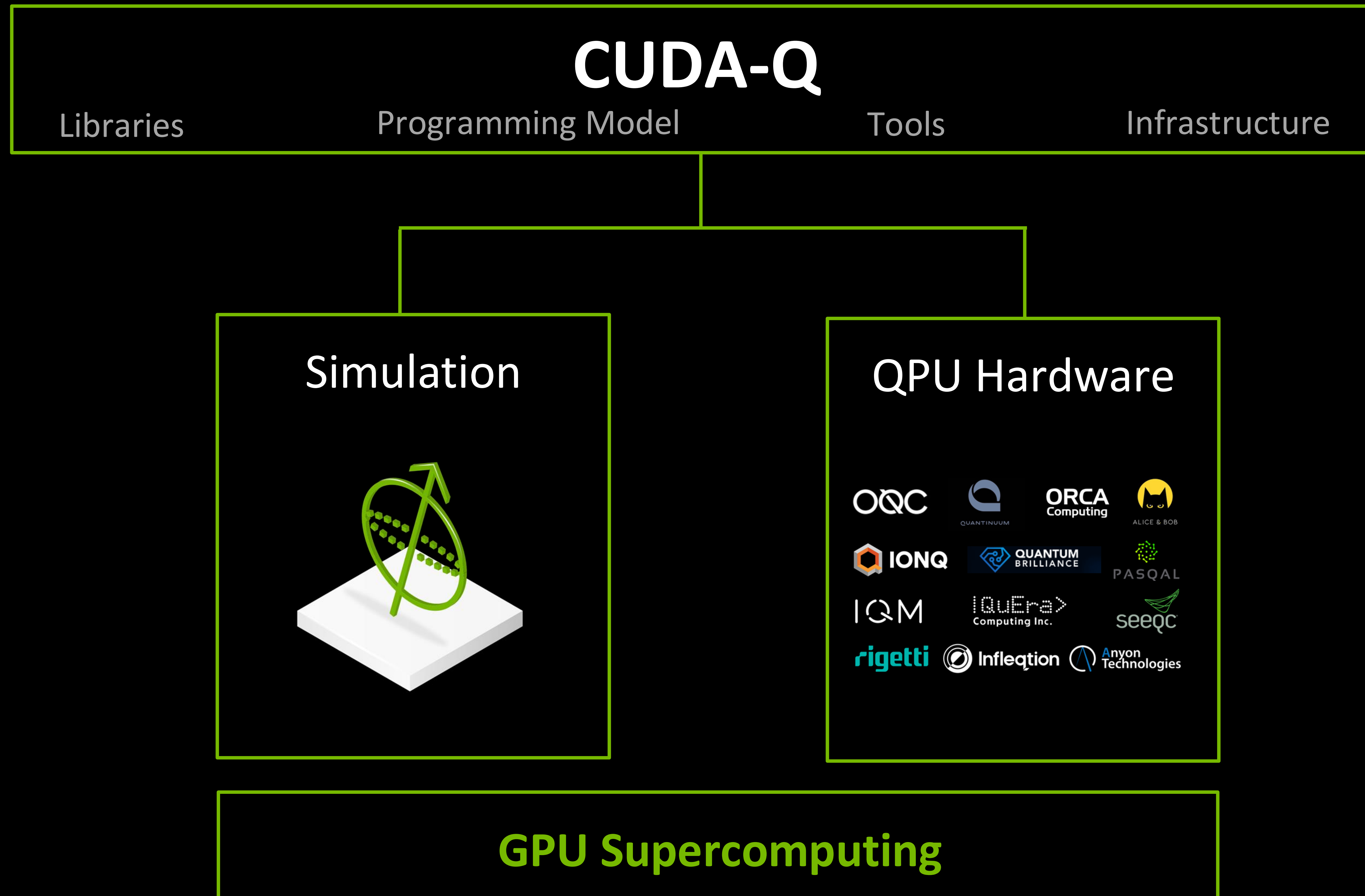
Petascale compute needed for useful Fault Tolerant Quantum Computers

Developer ecosystem is critical



CUDA-Q

The platform for accelerated quantum computing



CUDA-Q Python Bindings

Core C++ compiler platform exposed via builder API
JIT compile Quake representation at runtime

- Simple ways to create quantum kernels

```
import cudaq

cudaq.set_target('nvidia')

@cudaq.kernel
def kernel():
    # A single qubit initialized to the
    # ground / zero state
    qubit = cudaq.qubit()

    # Apply the Pauli x gate to the qubit
    x(qubit)

    # Measurement operator
    mz(qubit)

    # Sample the qubit for 1000 shots to
    # gather statistics
    result = cudaq.sample(kernel,
        shots_count=1000)
```

```
import cudaq

cudaq.set_target('quantinuum', machine='H2-2')

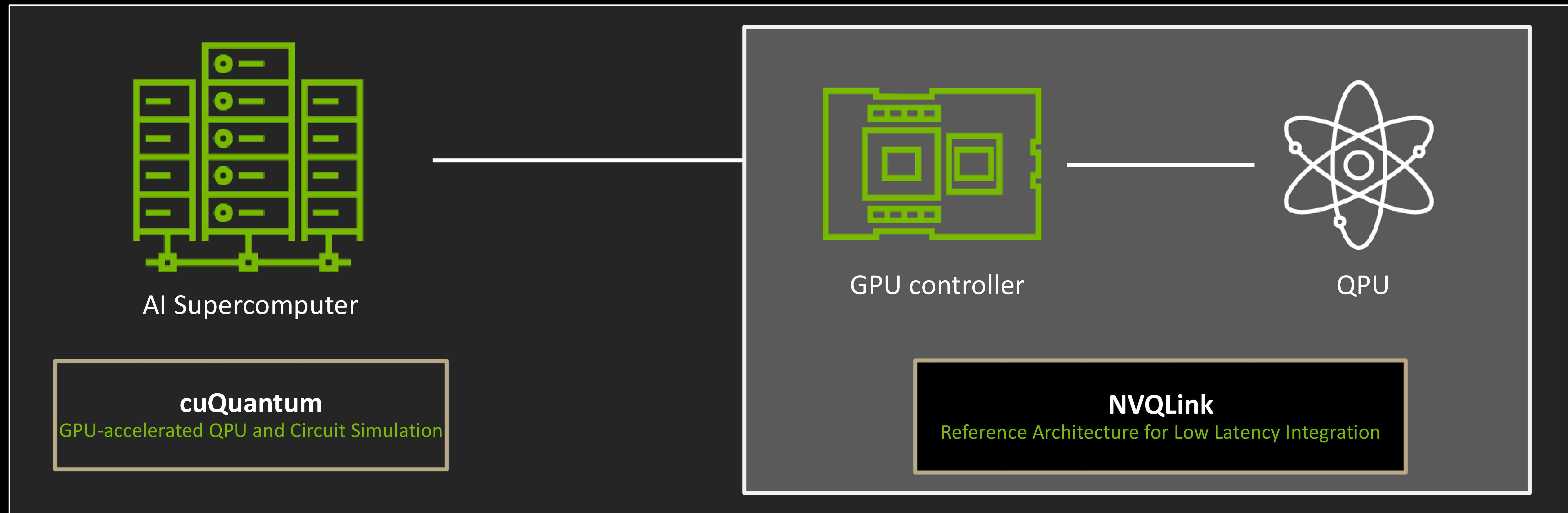
@cudaq.kernel
def kernel(angles: List[float]):
    # Allocate a qubit that is initialized to
    # the  $|0\rangle$  state.
    qubit = cudaq.qubit()

    # Apply parameterized gates to the qubit
    rx(angles[0], qubit)
    ry(angles[1], qubit)

    # Gate parameters which initialize the qubit
    # in the zero state
    initial_parameters = [0, 0]

    # Draw the circuit diagram for the kernel
    print(cudaq.draw(kernel, initial_parameters))
```

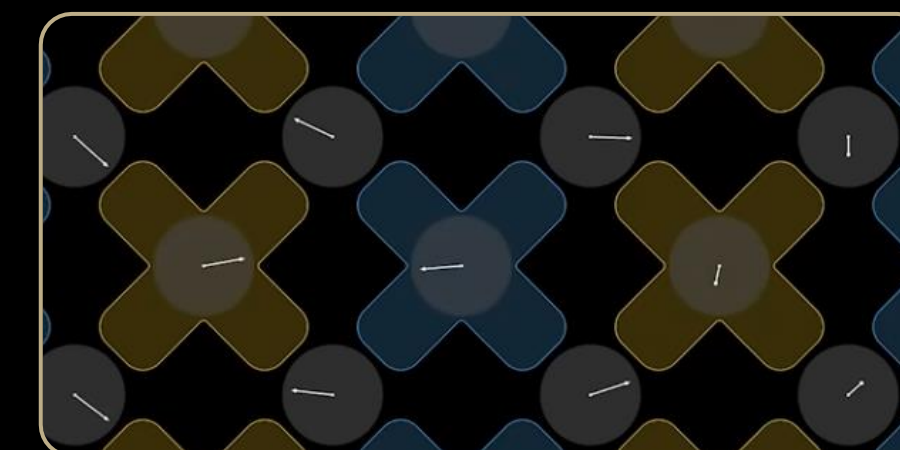
NVIDIA Open Quantum Platform



CUDA-Q Quantum-GPU Platform



CUDA-Q Solvers

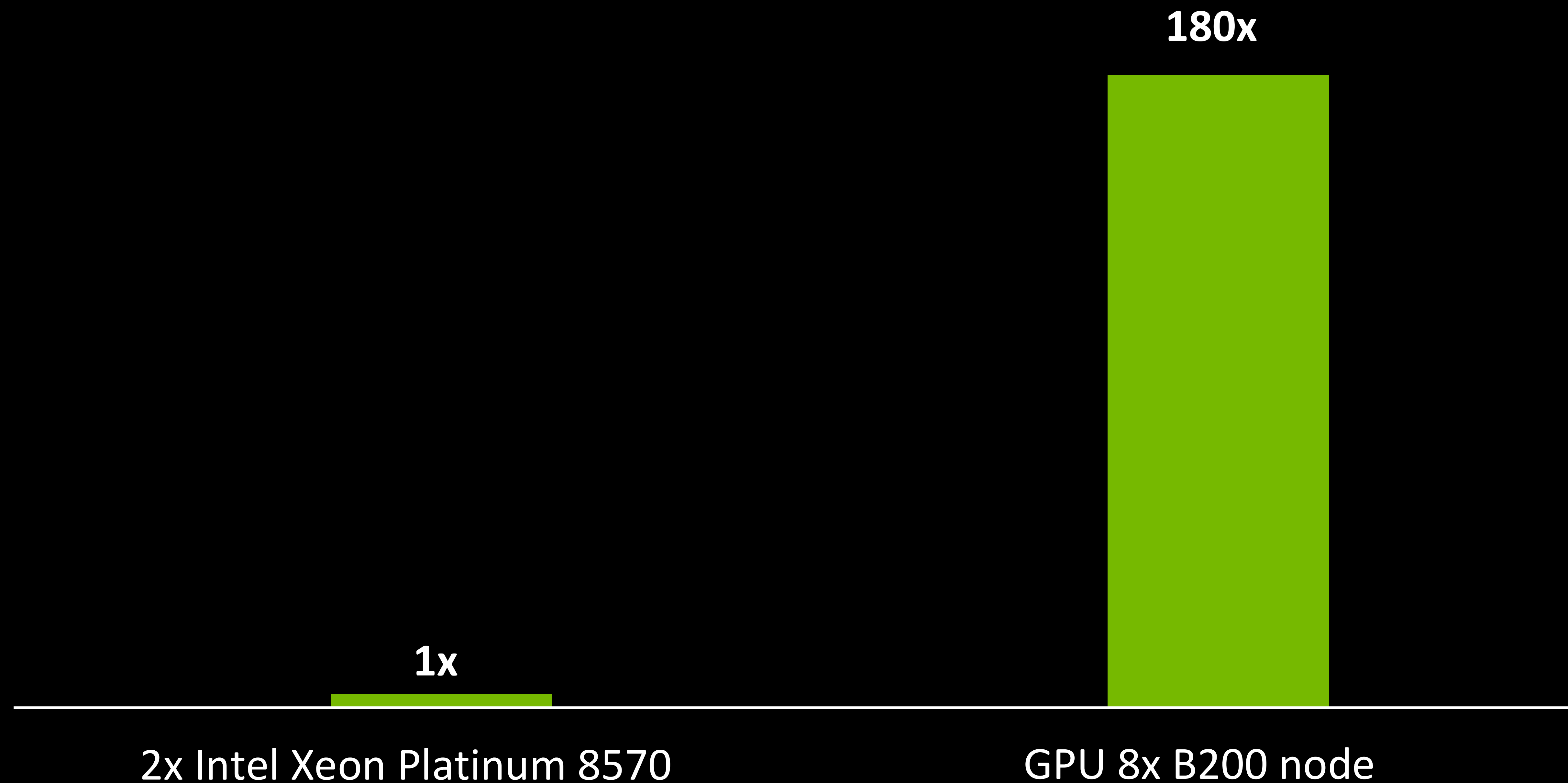


CUDA-Q QEC

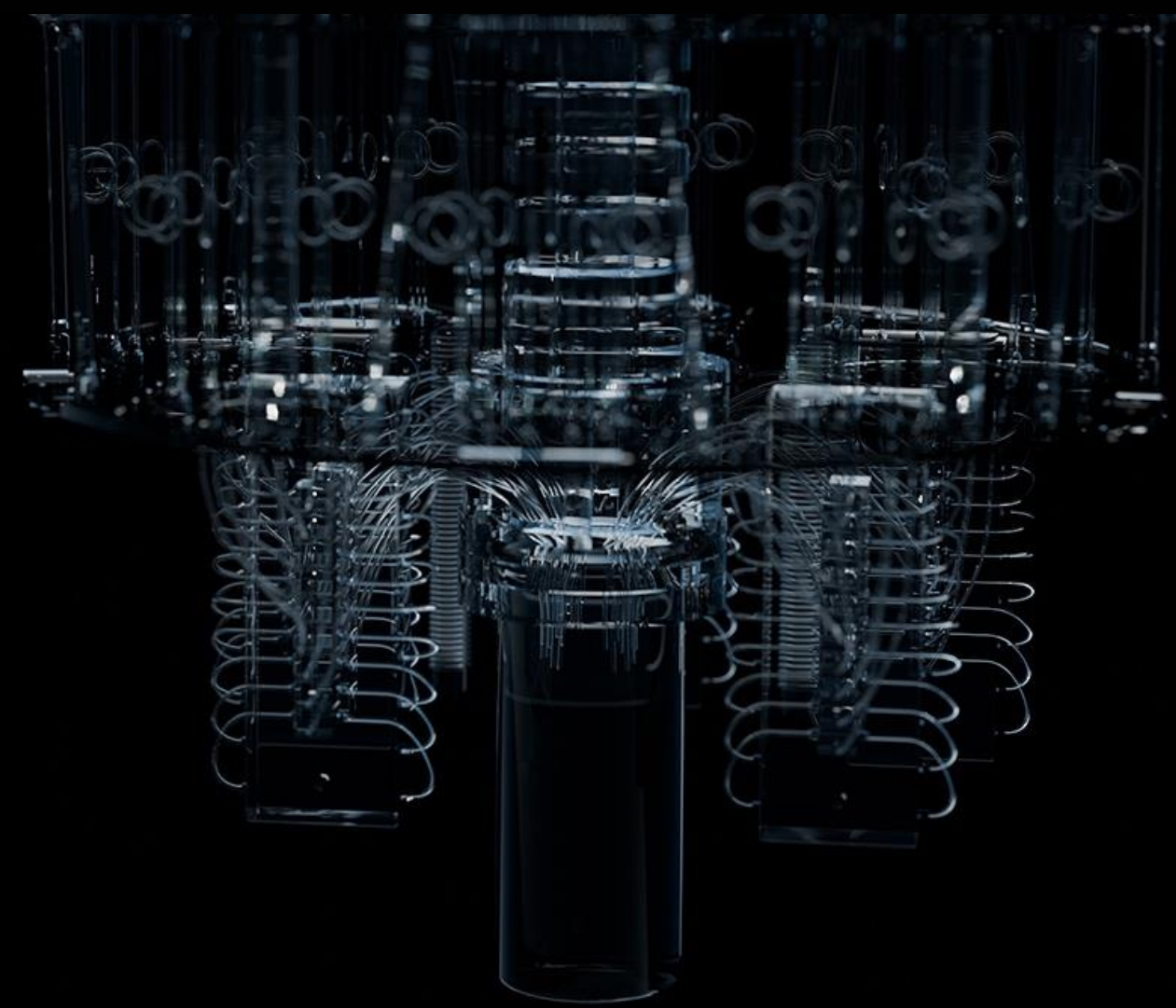
CUDA-Q GPU-Acceleration

CPU's are not enough for the simulations needed to develop quantum computing

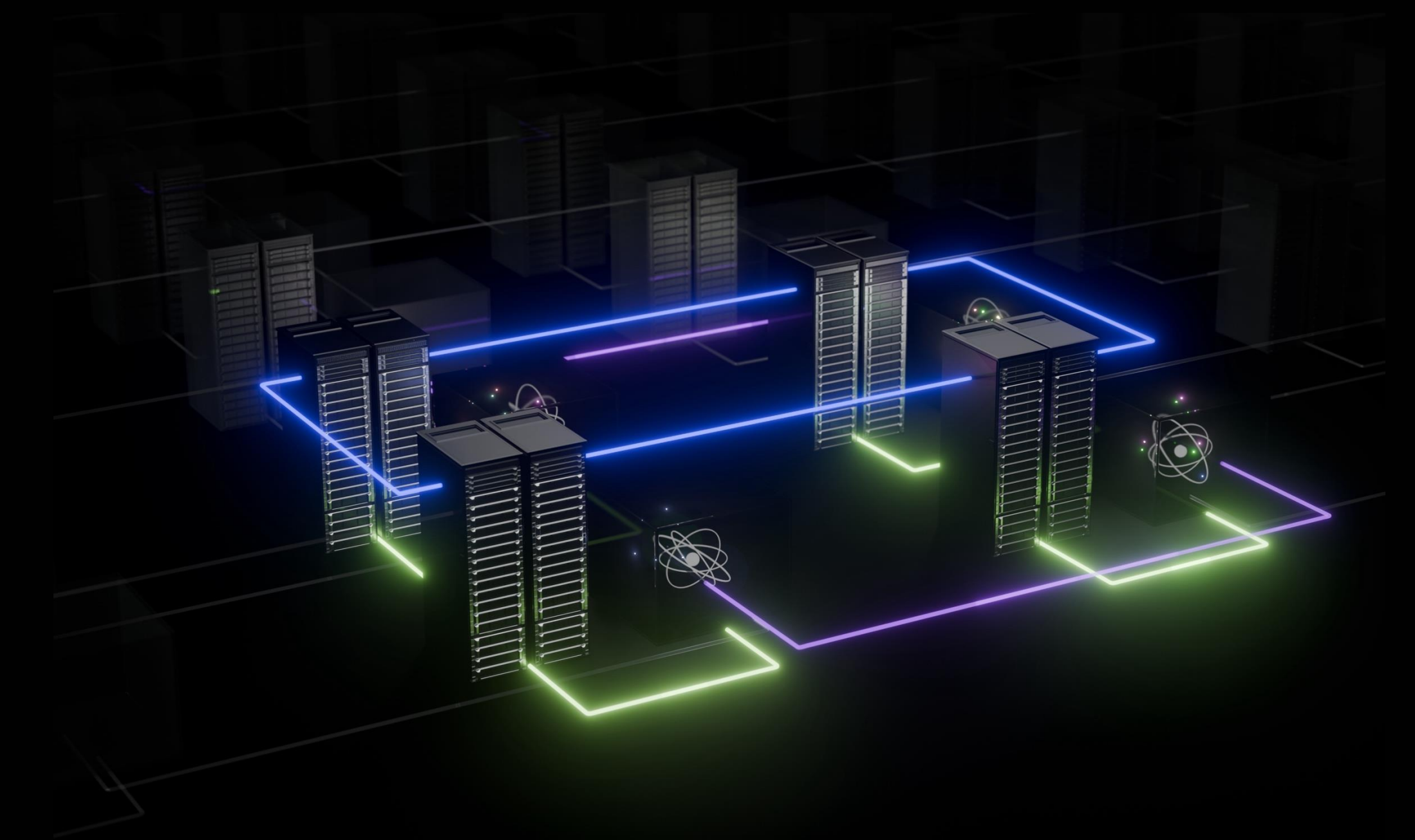
Speedup for 35 Qubit Quantum Volume Calculation



The Path from Qubits to Supercomputers



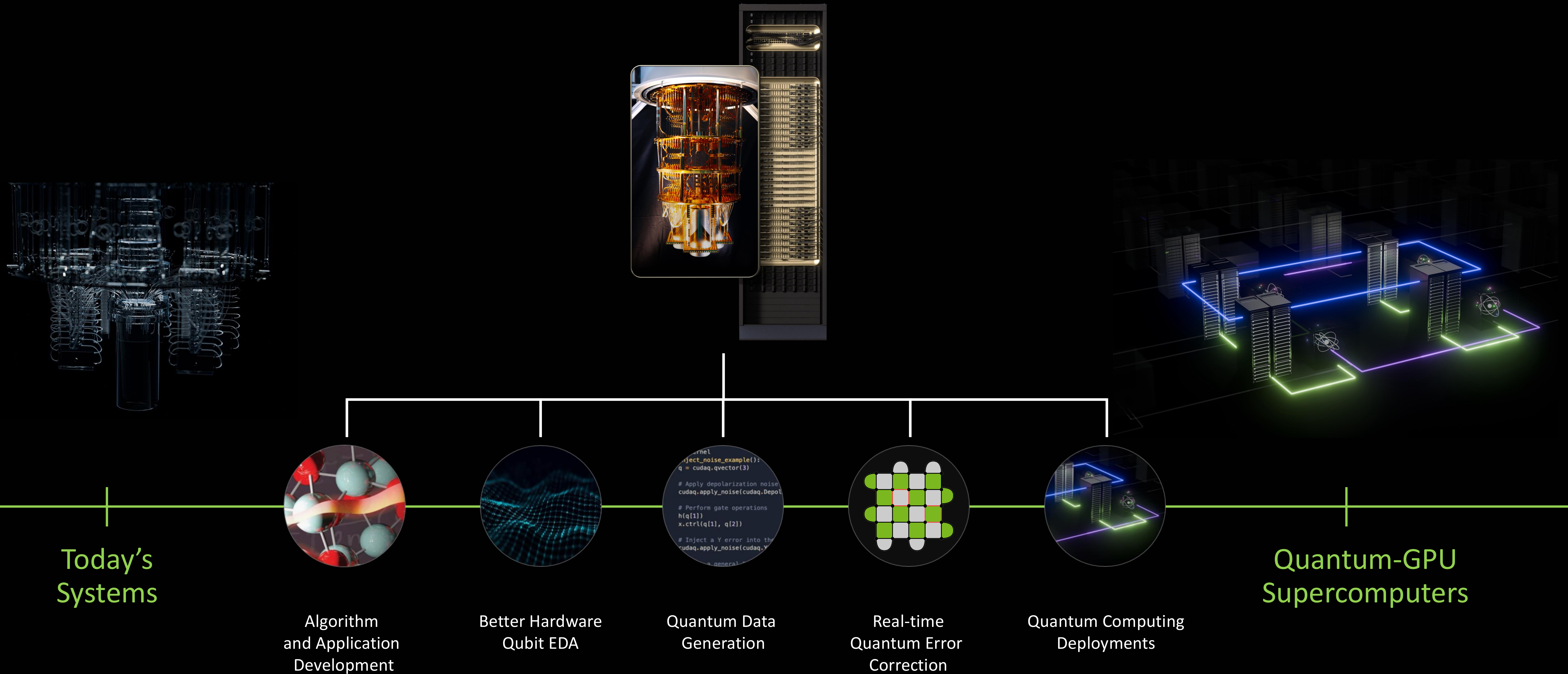
Today's
Systems



Quantum-GPU
Supercomputers

The Path from Qubits to Supercomputers

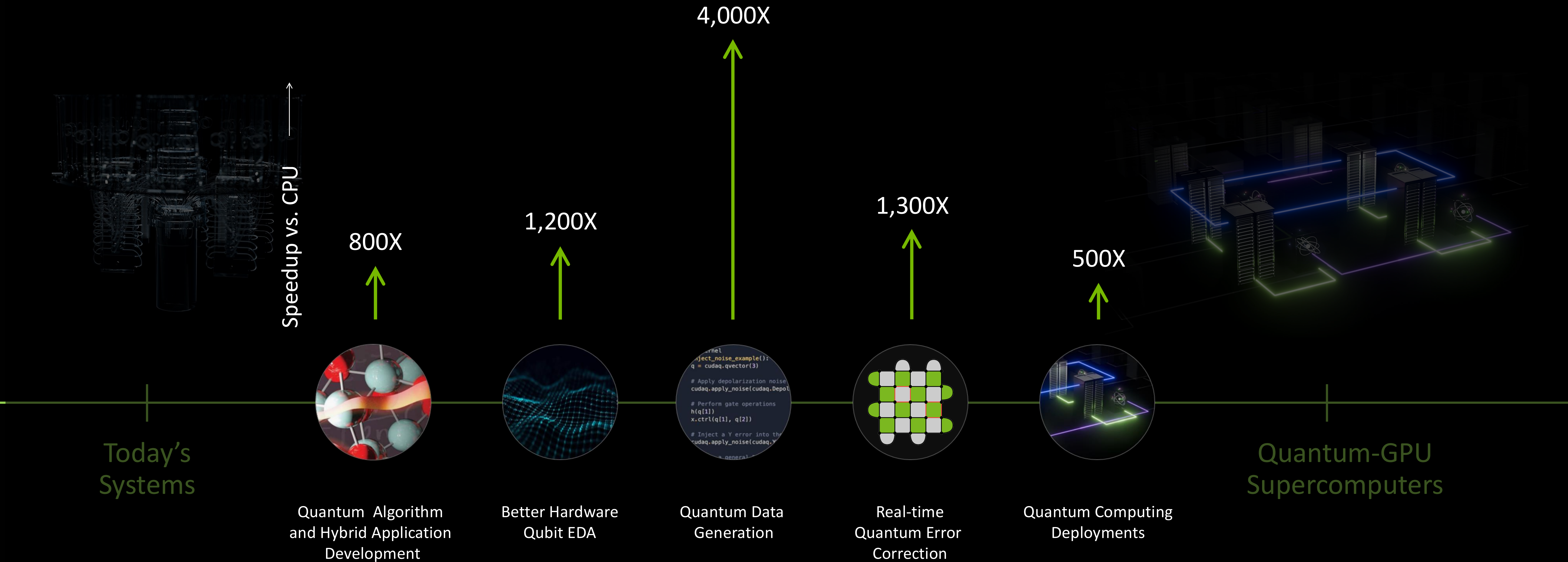
AI to the Rescue



Alexeev, Y., Farag, M.H., Patti, T.L. *et al.* Artificial intelligence for quantum computing. *Nat Commun* **16**, 10829 (2025). [arXiv:2411.09131](https://arxiv.org/abs/2411.09131)

The Path from Qubits to Supercomputers

AI to the Rescue



Alexeev, Y., Farag, M.H., Patti, T.L. *et al.* Artificial intelligence for quantum computing. *Nat Commun* **16**, 10829 (2025). [arXiv:2411.09131](https://arxiv.org/abs/2411.09131)

DGX SuperPOD Successes for Quantum

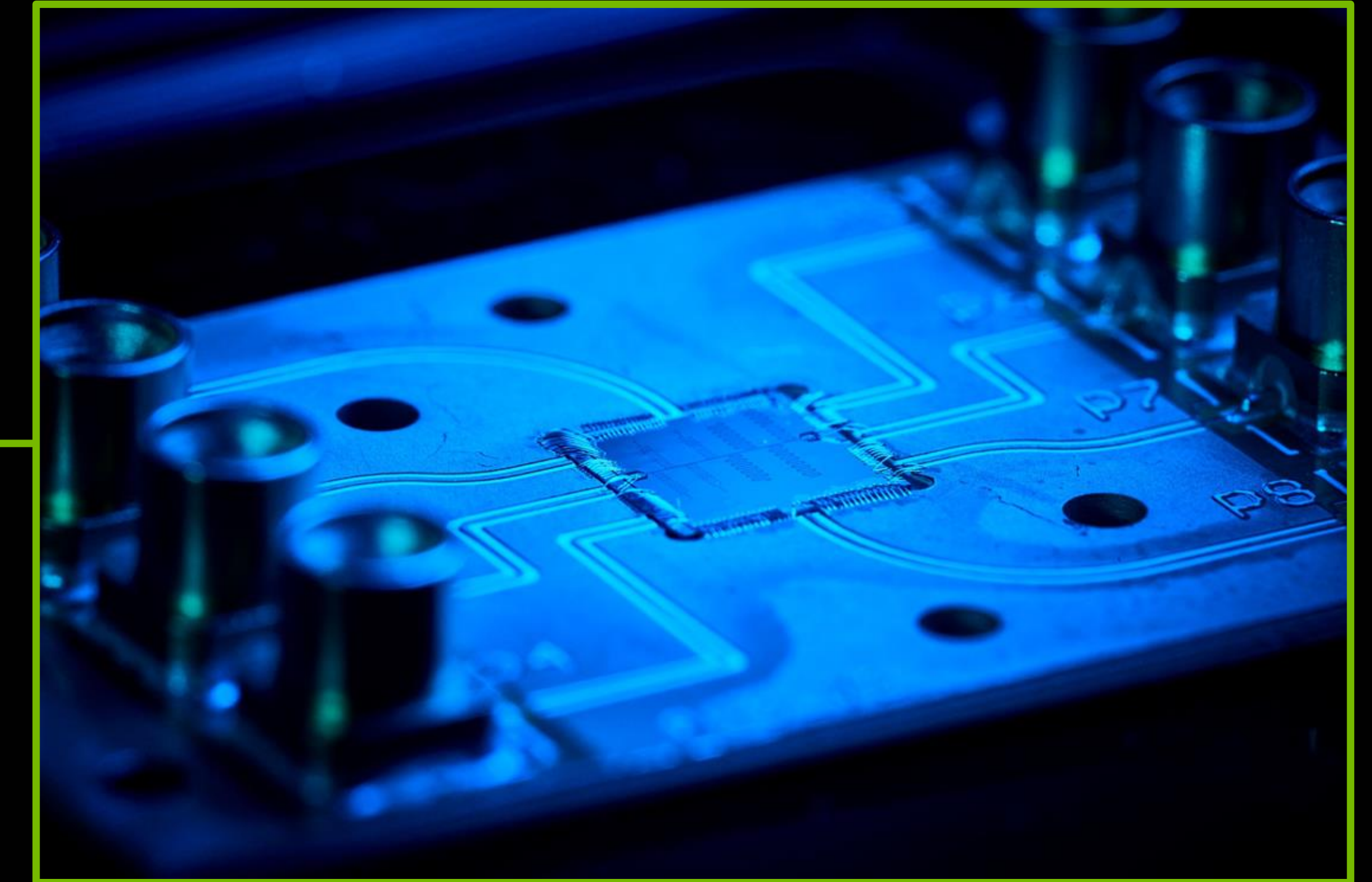
“Gefion will be an essential tool to support our full stack quantum computing research as part of the NQCP - NNF’s 12- year initiative...”

- Peter Krogstrup, CEO NNF QC Program

Gefion

**Novo Nordisk
Foundation**

H100 + Infiniband



“ABCI-Q will let Japanese researchers explore quantum computing technology to test and accelerate the development of its practical applications.”

- Masahiro Horibe, deputy director of AIST

ABCI-Q

AIST and Fujitsu
>2,000 H100 + Infiniband



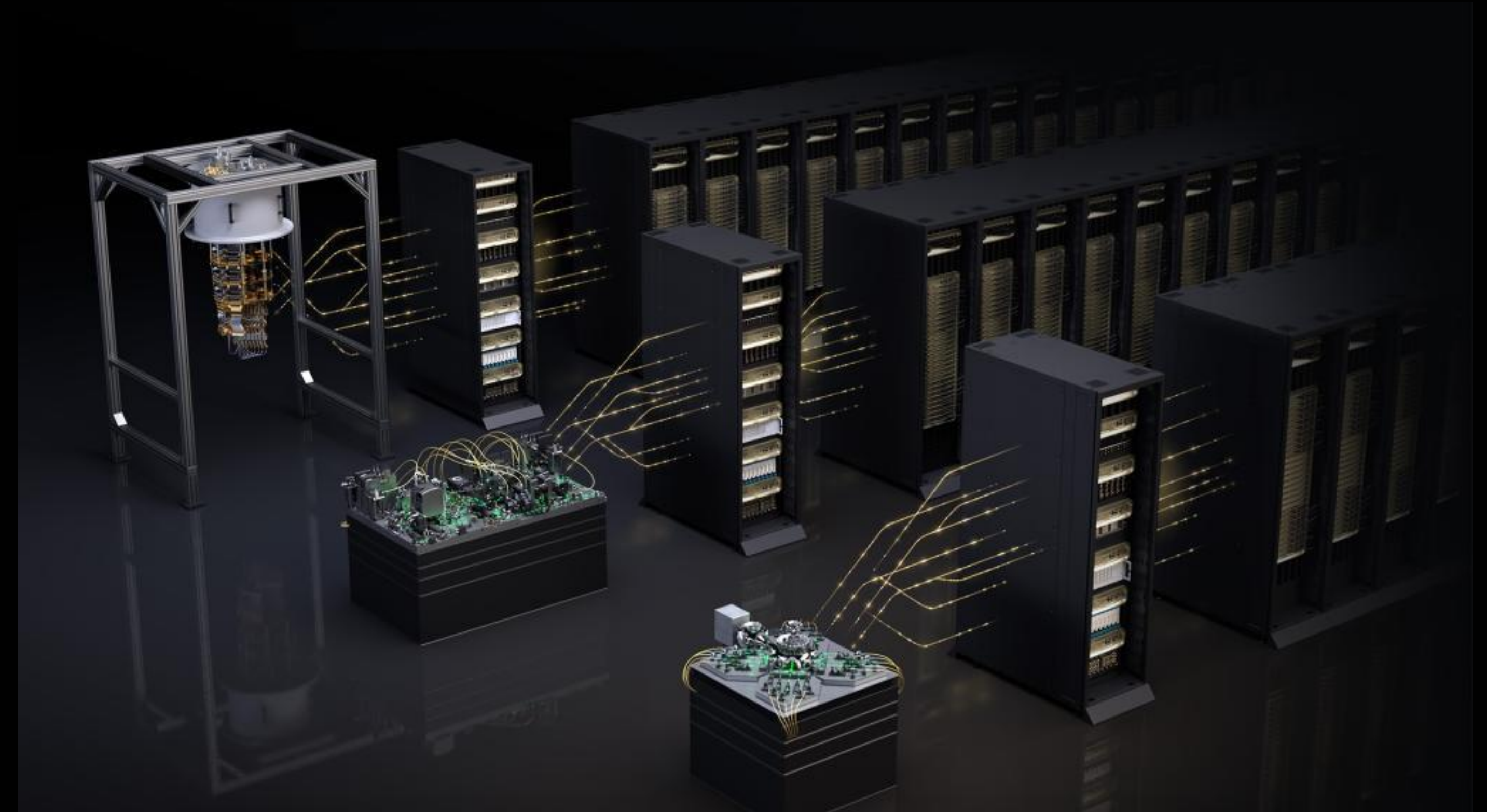
Quantum-GPU Supercomputing

New Deployments Building Quantum Ecosystems Worldwide

RIKEN Quantum



ORNL Quantum



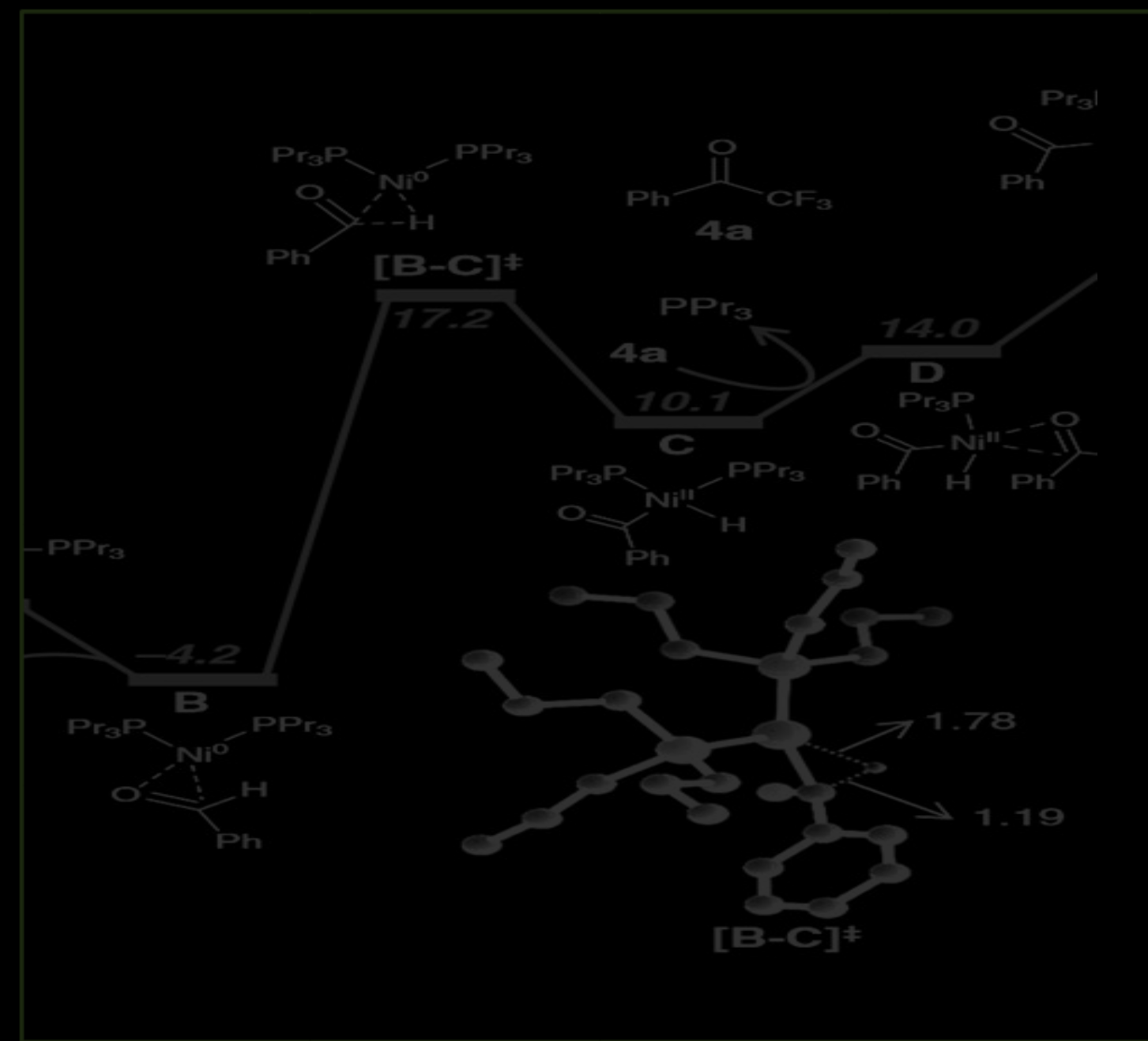
“With the introduction of this NVIDIA GPU-based system, we can...expand computational resources to support quantum-HPC hybrid computing ...”

Dr. Mitsuhsa Sato, Director R-CCS Quantum-HPC Hybrid Platform Division

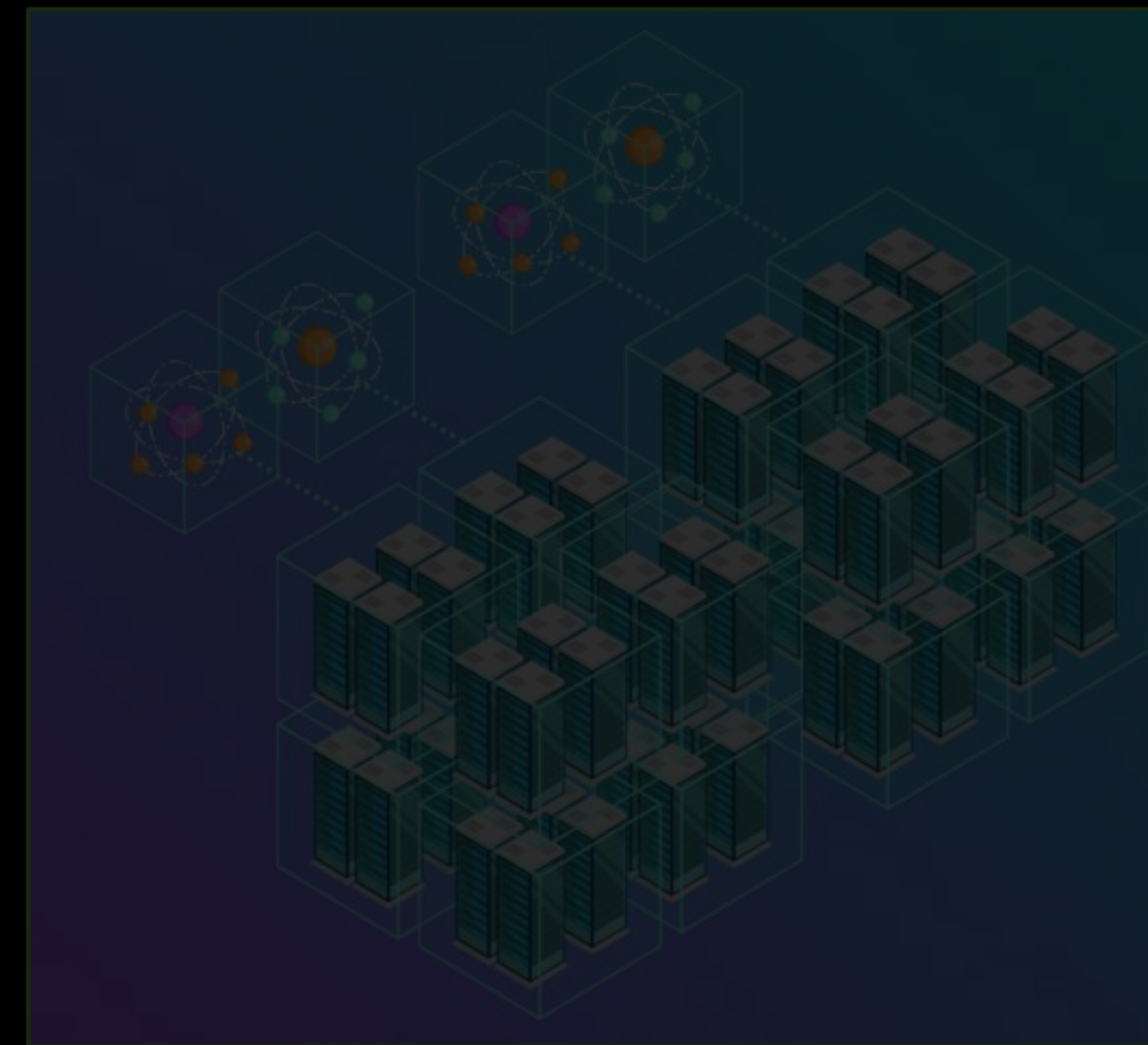
“...NVIDIA NVQLink provides the critical technology to unite world-class GPU supercomputers with emerging quantum processors, creating the powerful systems we need to solve the scientific challenges of our time.”

Chris Wright, U.S. Secretary of Energy

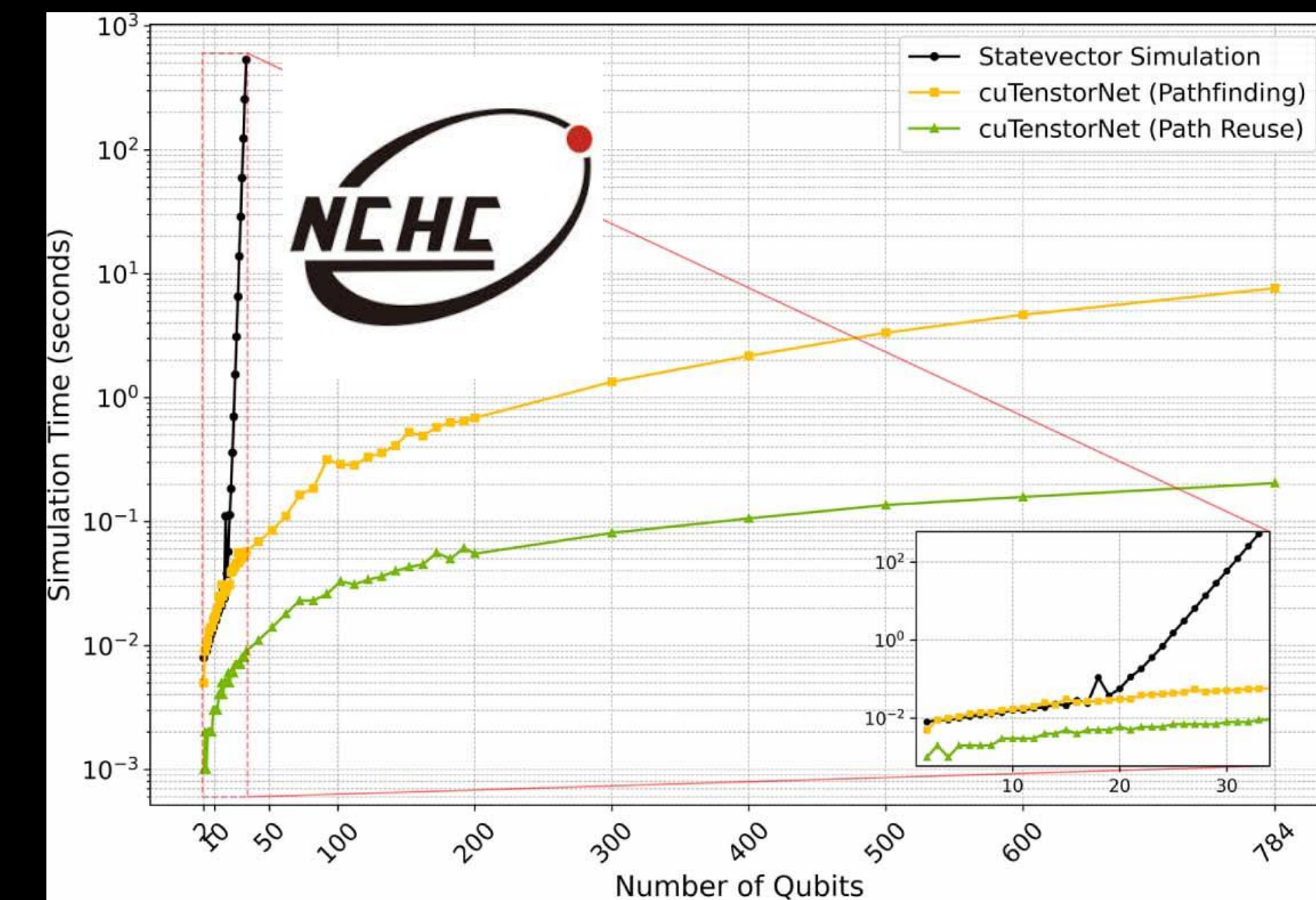
Better Algorithms with Large-Scale Simulations and AI



IonQ/AWS/AstraZeneca
AF-QMC simulation for
electronic structure modeling



HPE Labs
Distributed QC with
Adaptive Circuit Knitting



NCHC
> 1000-qubit simulation to
validate QML approaches

Qubits

Better QPUs

Better Algorithms

Scaling QEC

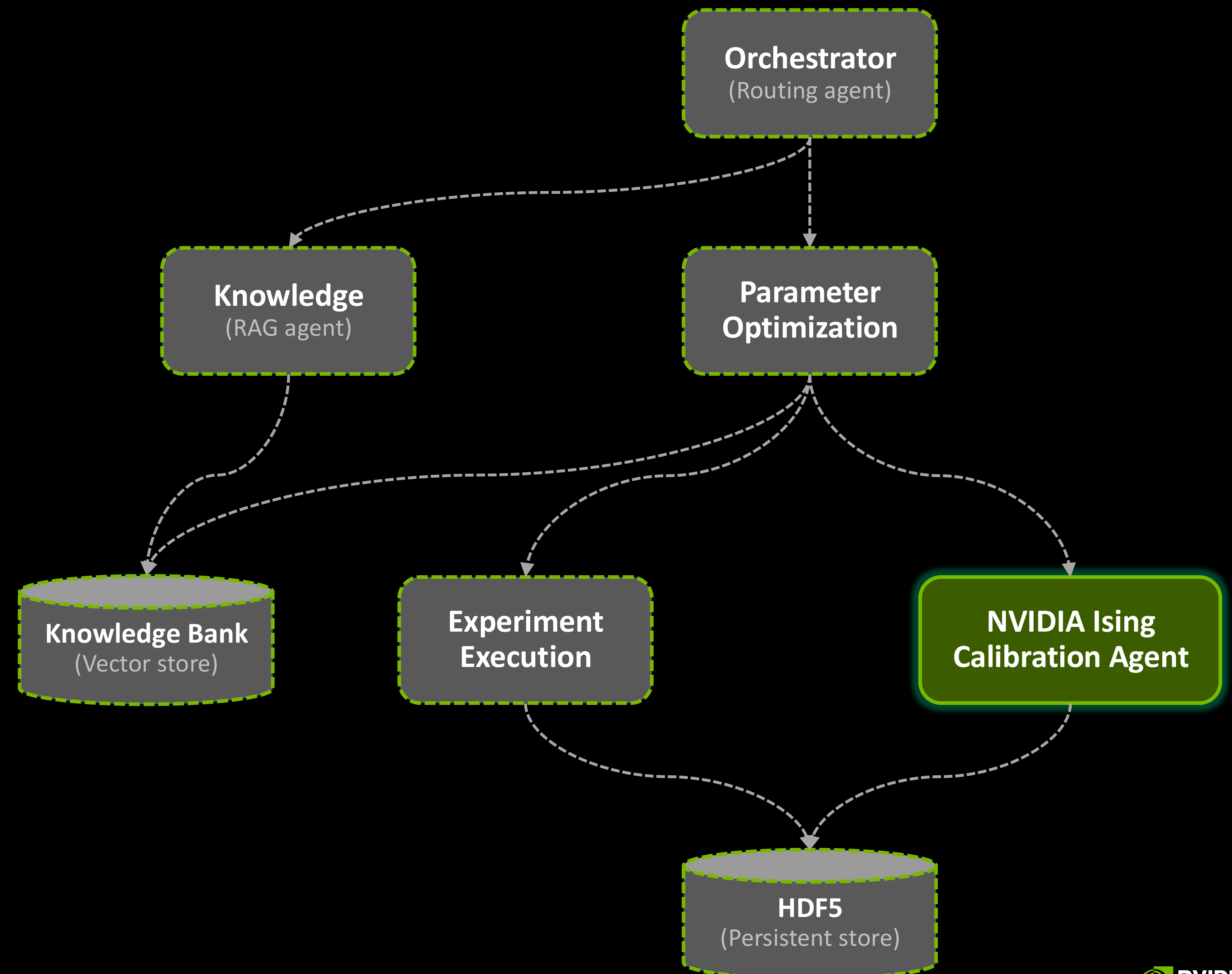
Accelerated Quantum
Supercomputers

Academia Sinica and NCHC Deploying Ising Calibration

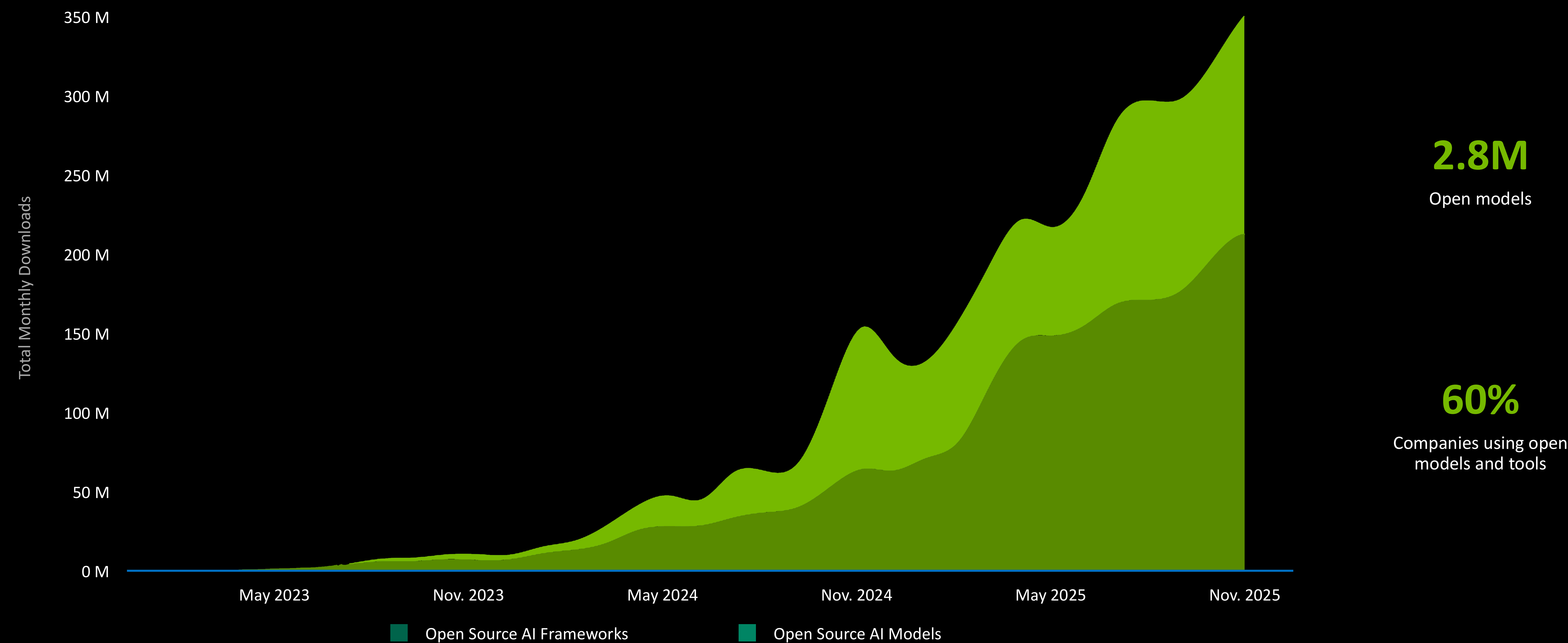


- **NVIDIA Ising** integrated into agentic calibration workflow
- Calibration performed via **NVQLink** interconnect with **NVIDAI GH200** and **Quantum Machines** controller
- Fully automated qubit spectroscopy with a **single prompt**

Hours → Minutes



NVIDIA Is Driving Innovation With Open Models



NVIDIA Ising

World's First Open Models Accelerate Path to Useful Quantum Computers



- The open platform for all QPU developers to build on
- Provides open access, fine-tuning and data sovereignty
- Offers leading performance in QPU calibration and decoding
- Scales to large-scale quantum-GPU supercomputing

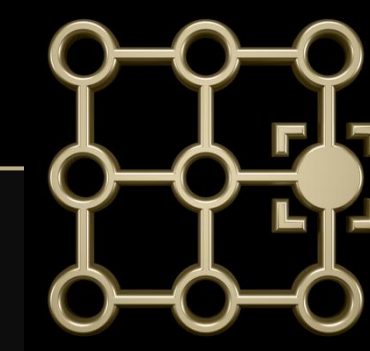
NVIDIA Ising

World's First Open Models Accelerate Path to Useful Quantum Computers



Ising Calibration

Vision Language Model (VLM)



Ising Decoding

Convolutional Neural Network (CNN)

Cookbooks

Fine-Tuning, Quantization, Inference
Workflows

Open Research

Open Models and
Papers

Open Data

Benchmark Data and Training

NVIDIA Ising

World's First Open Models Accelerate Path to Useful Quantum Computers

Ising Calibration



atom
computing



Conductor
Quantum



Fermilab



Infleqtion



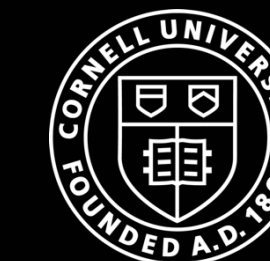
IQM



Q-CTRL



Ising Decoding



Infleqtion

IQM



UCSB

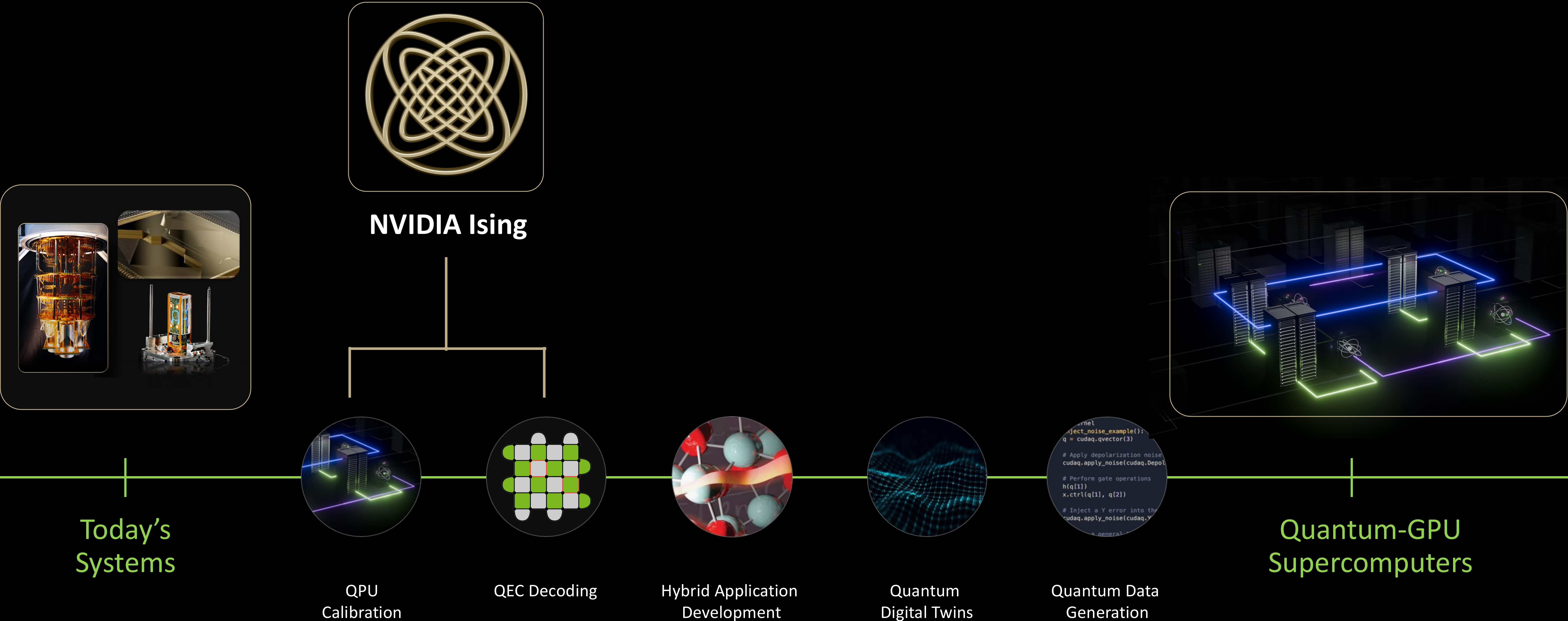
seeqc



UC San Diego
JACOBS SCHOOL OF ENGINEERING

NVIDIA Ising

Open Models For QPU Calibration and QEC Decoding

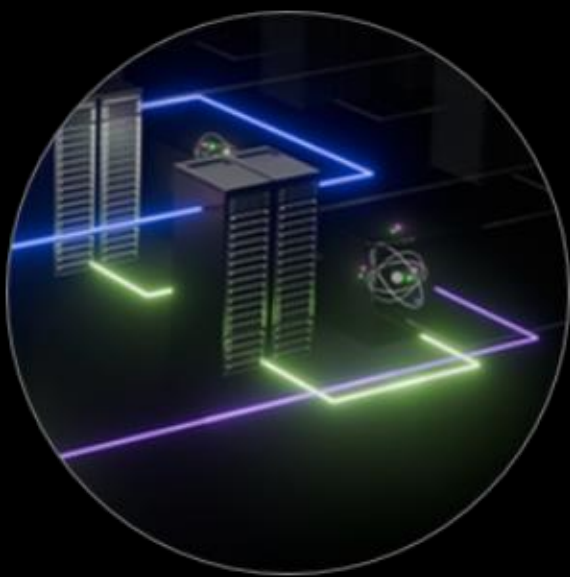
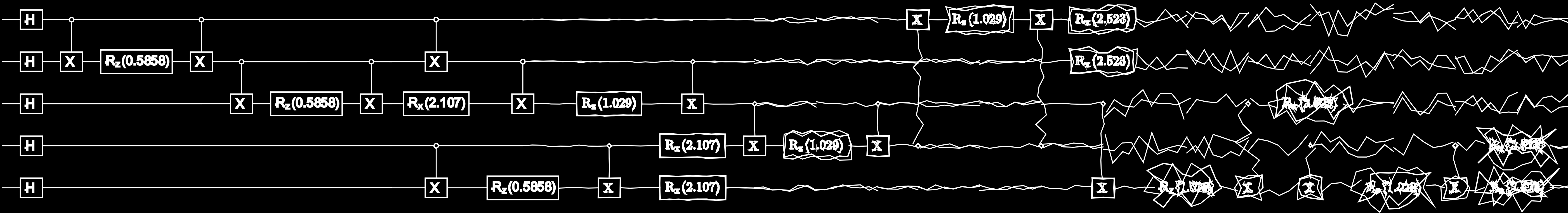


NVIDIA Ising

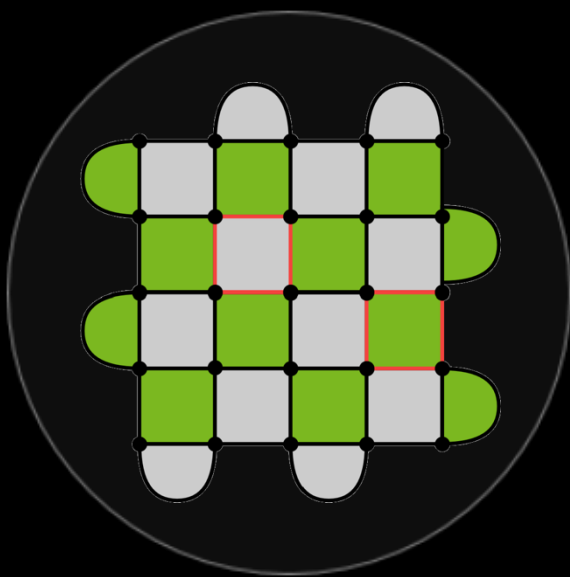
Open Models For QPU Calibration and QEC Decoding



NVIDIA Ising

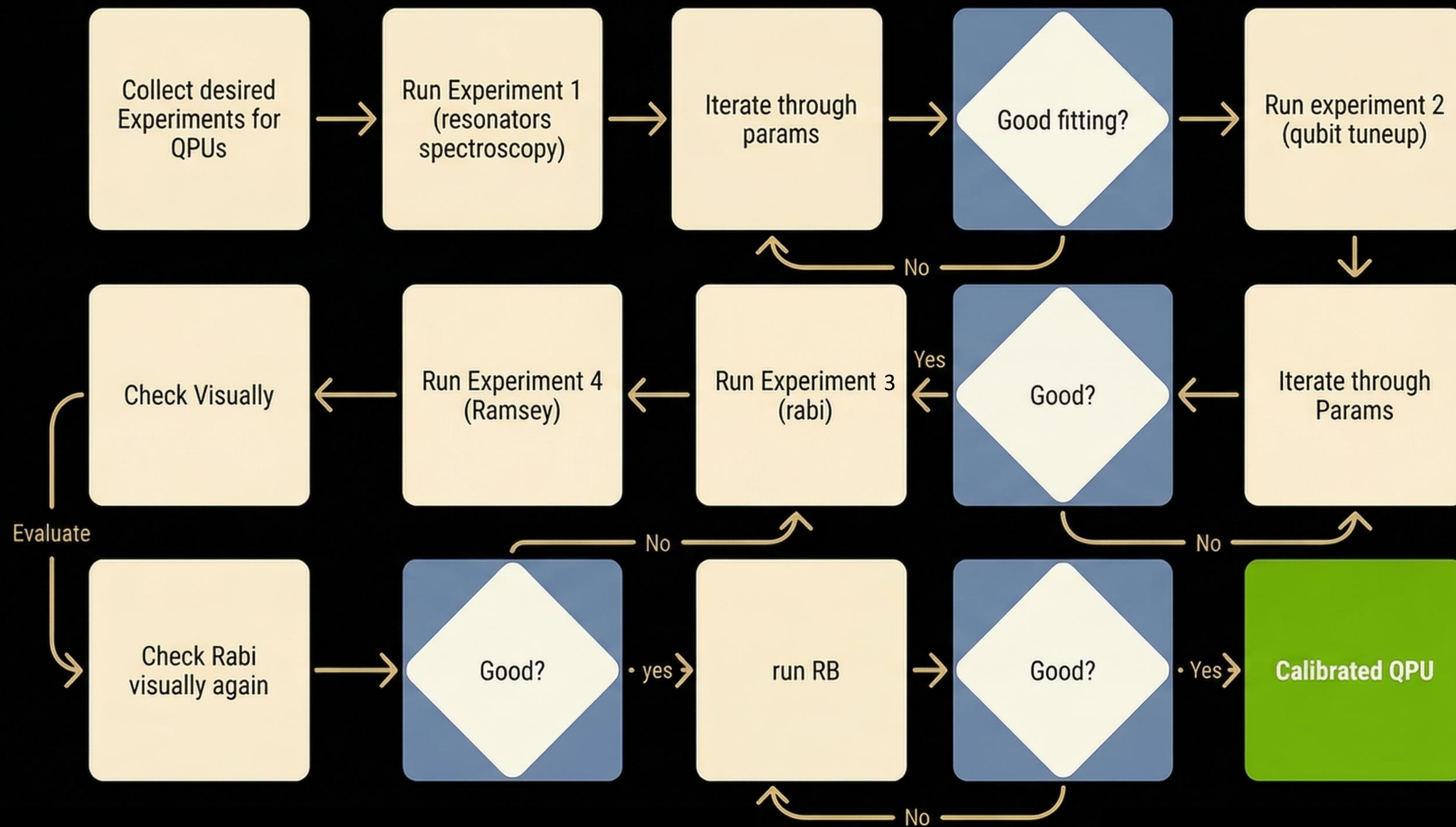


QPU
Calibration



QEC Decoding

Existing QPU Calibration is a Complex, Manual Process

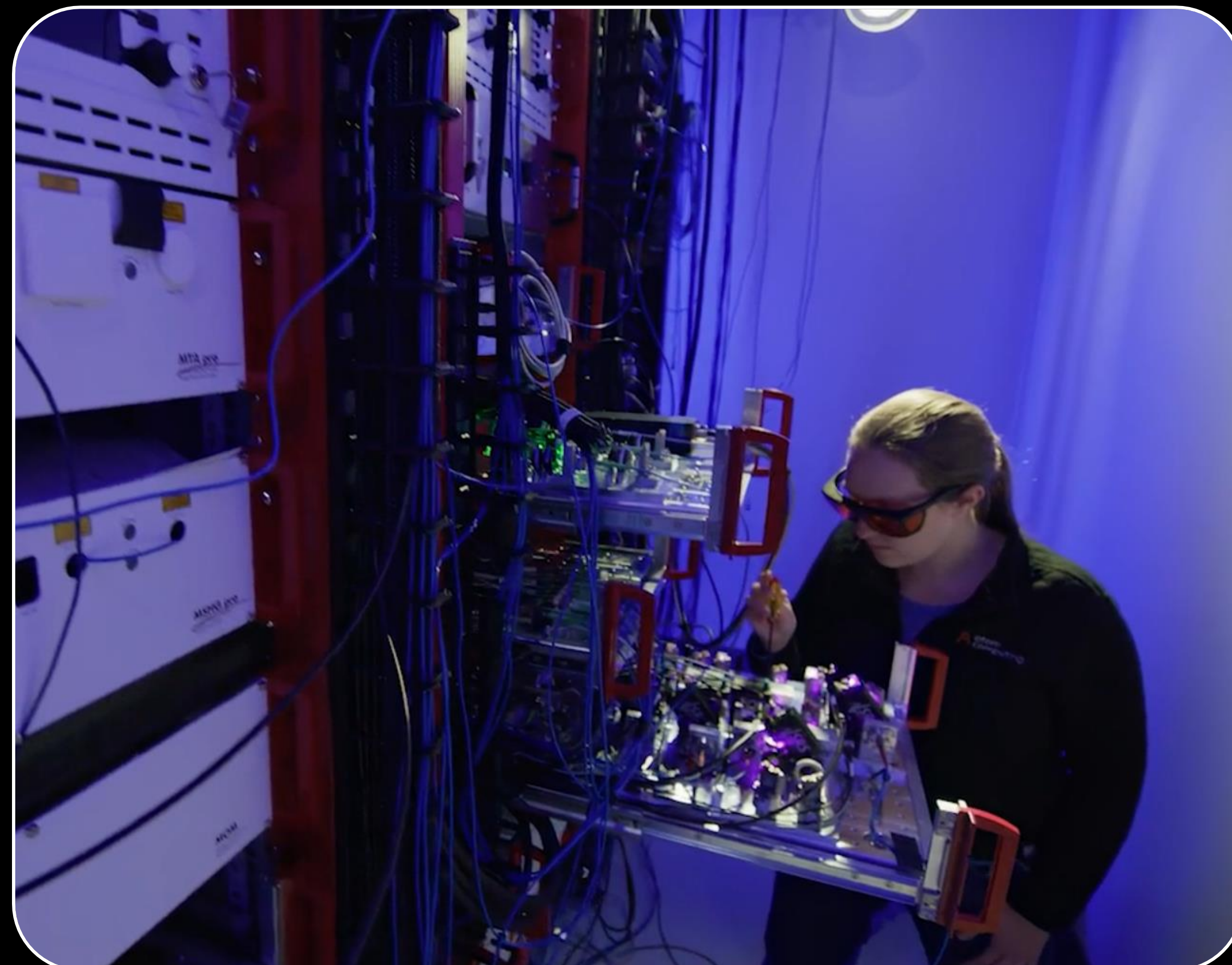


NVIDIA Ising Calibration

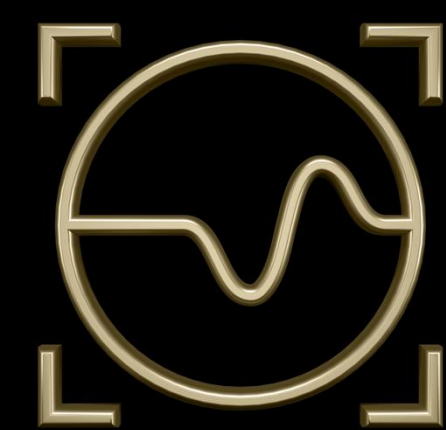


Ising Calibration

Vision Language Model (VLM)



- VLM for quantum calibration plot understanding
- Agentic workflow for automatically calibrating QPUs
- QPU-agnostic, qubit performant
- Outperforms any other model for calibration

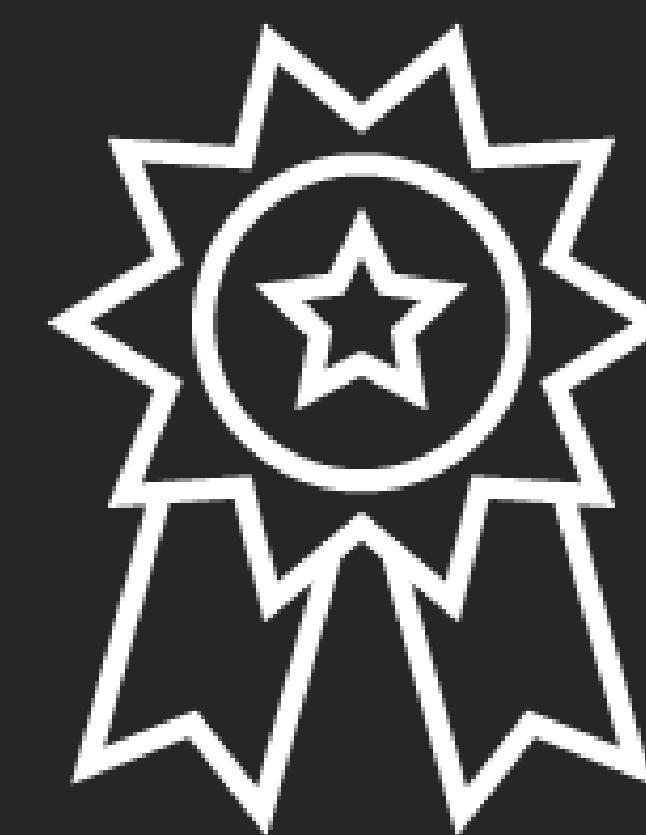


NVIDIA Ising Calibration



Days → Hours

Calibration Time
Human versus AI Agent

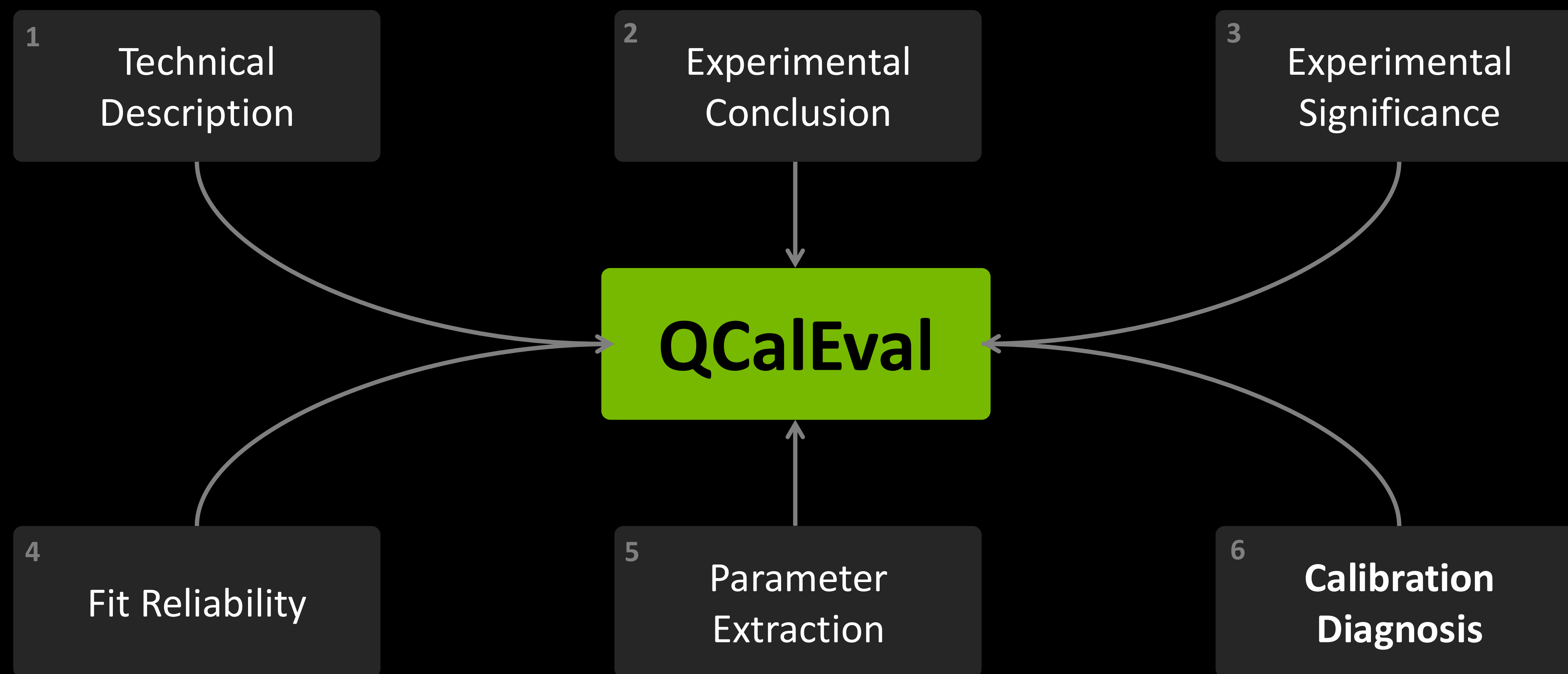


World's Best

On QCalEval Calibration Benchmark

QCalEval

World's First Calibration Benchmark

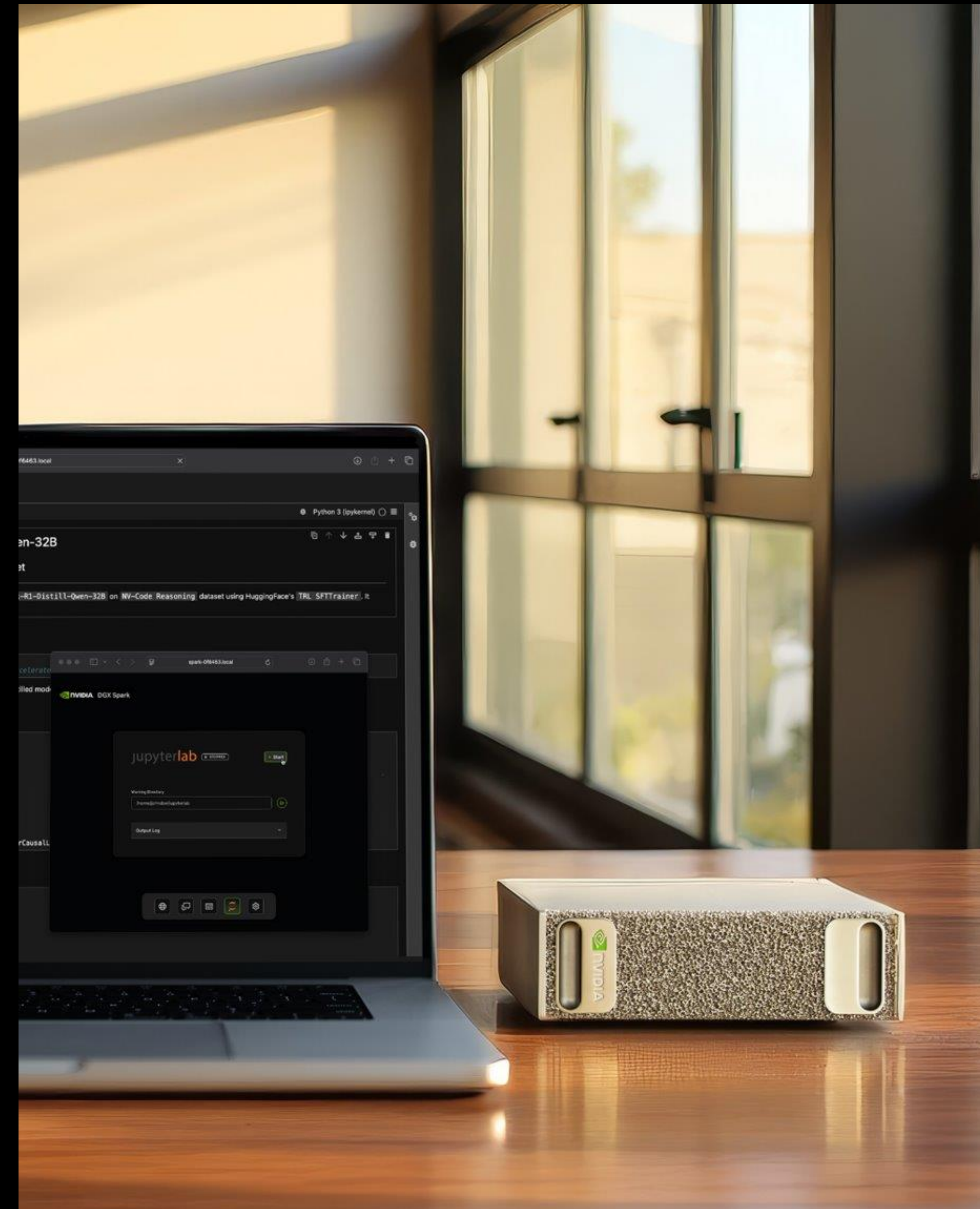


Created with data from
7 QPU Builders

Validated by
20+ Quantum Partners

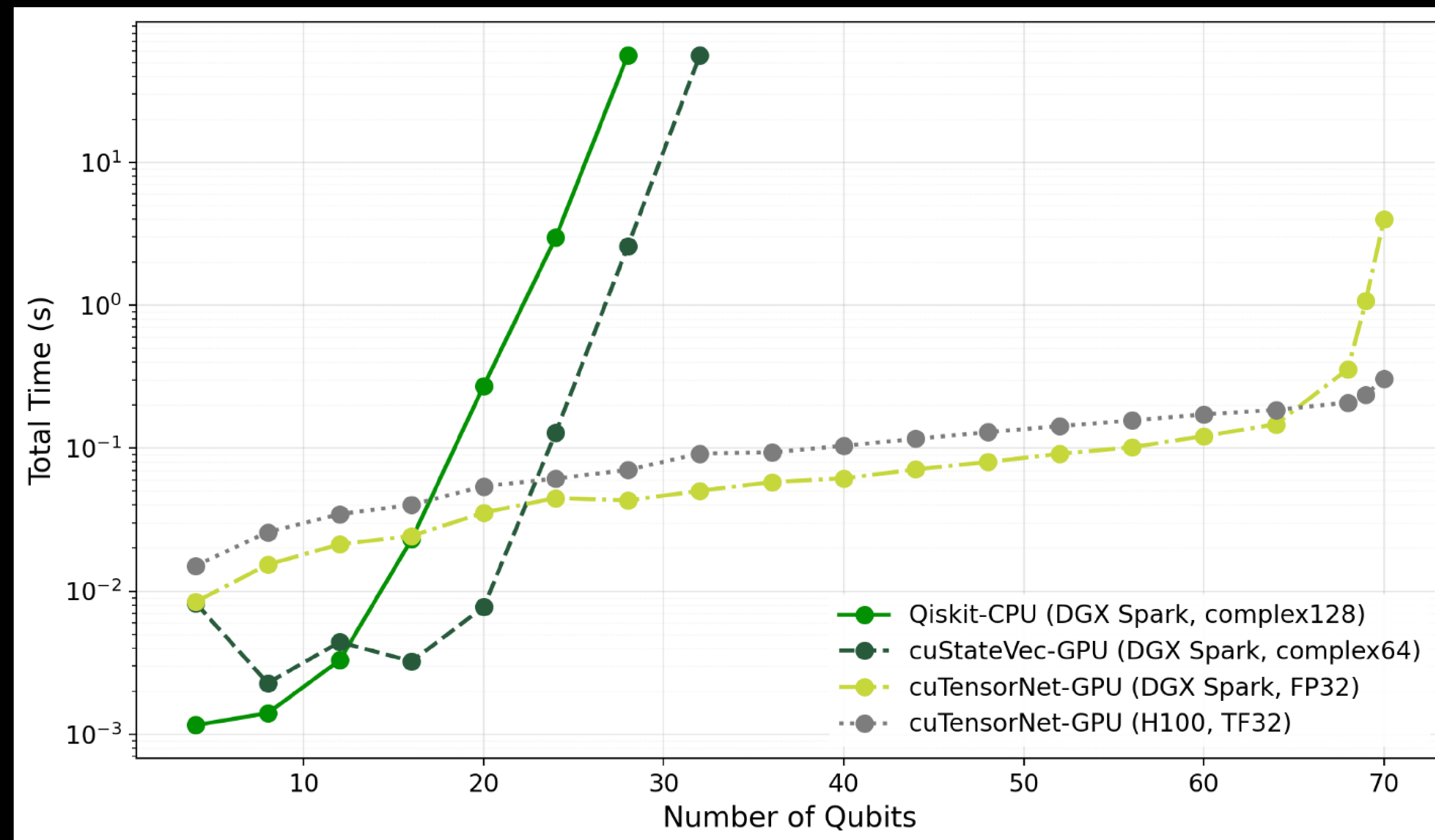
DGX Spark for Quantum Computing

- NVIDIA **Grace Blackwell Architecture**
- **QPU simulation**
 - Full state vector simulation up to 32 qubits
 - Tensor network simulation for larger systems
- Train and run **AI models for quantum** (AI decoders, quantum EDA, circuit design and more)
- Compact, power efficient form factor
- Configure as a simulated **desktop QPU** or as a **GPU resource in a hybrid quantum-GPU setup**



DGX Spark for Quantum Computing

The supercomputer in the lab



FAST-VQE benchmarking on DGX Spark with cuQuantum (cuTensorNet) acceleration

- NVIDIA **GB10 Grace Blackwell superchip**
- **1 PFLOP of FP4** AI performance
- **128GB** coherent, unified system memory

- NVQLink-compatible **ConnectX-7 Smart NIC**
- 150mm L x 150mm W x 50.5mm H

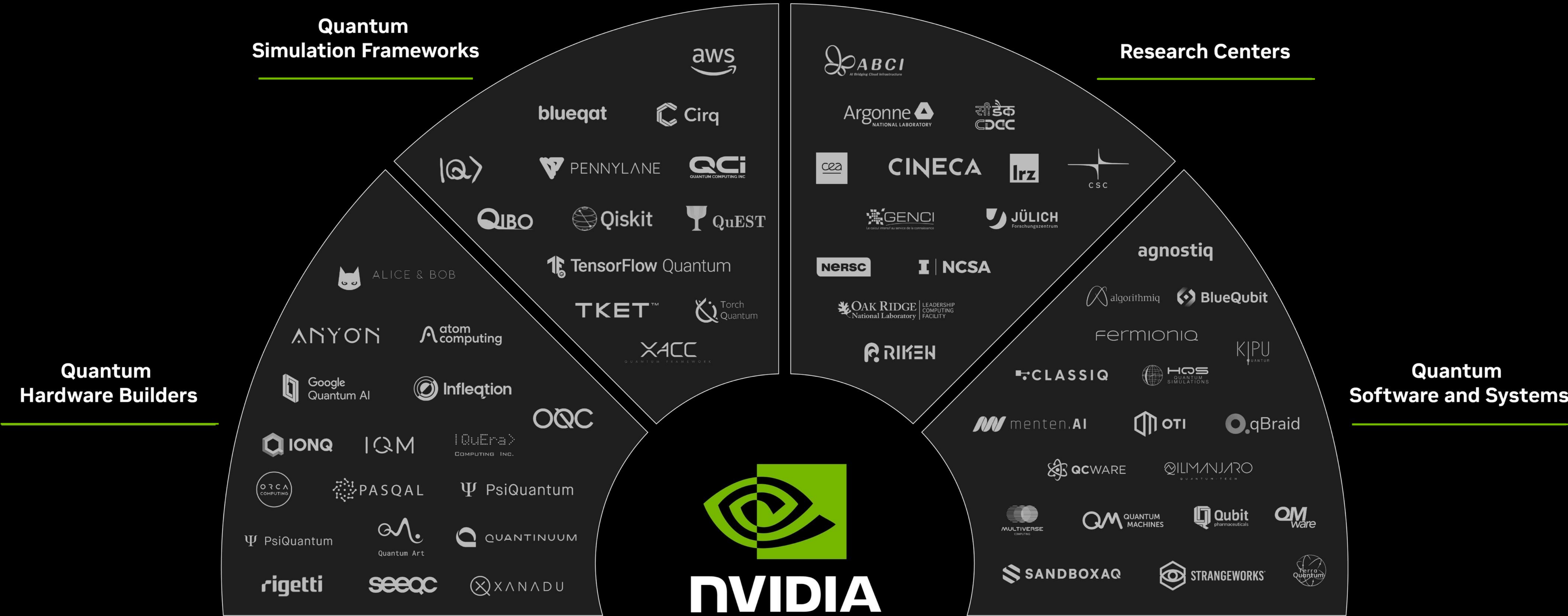
NVIDIA Open Quantum Platform

200+
NVIDIA Quantum Partners

>90%
Largest Startups
Working with NVIDIA

>80%
QPUs Integrating
NVIDIA Software

100%
Leading Quantum
Development Frameworks
Accelerated



CUDA-Q Now Available

The hybrid quantum computing platform

CUDA-Q Overview

<https://developer.nvidia.com/cuda-q>

CUDA-Q Academic

Educational resources for CUDA-Q
(<https://github.com/NVIDIA/cuda-q-academic>)

CUDA-Q Applications

<https://nvidia.github.io/cuda-quantum/latest/using/tutorials.html>



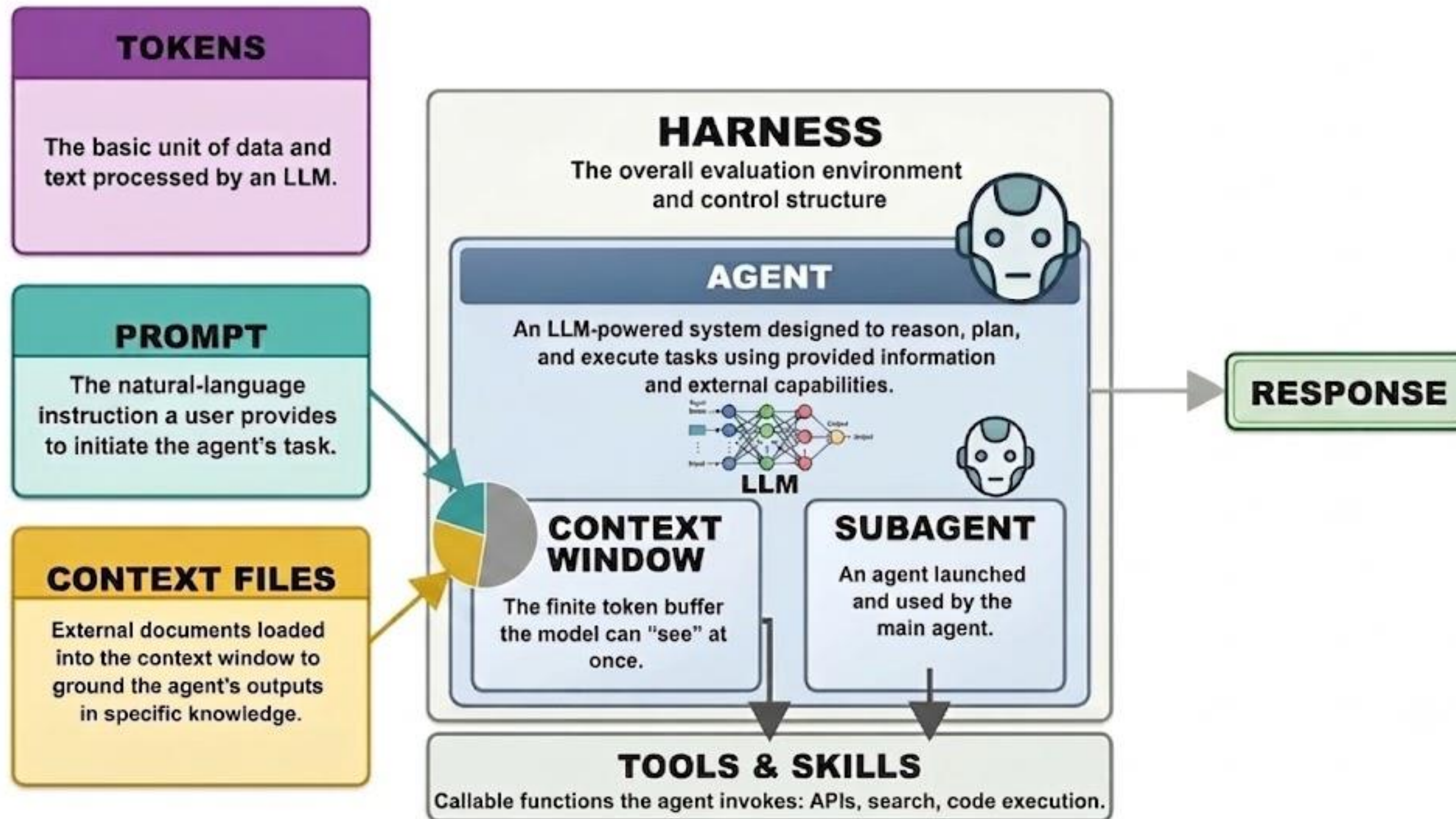


Hands-on

Foundations of Agentic Coding Coding

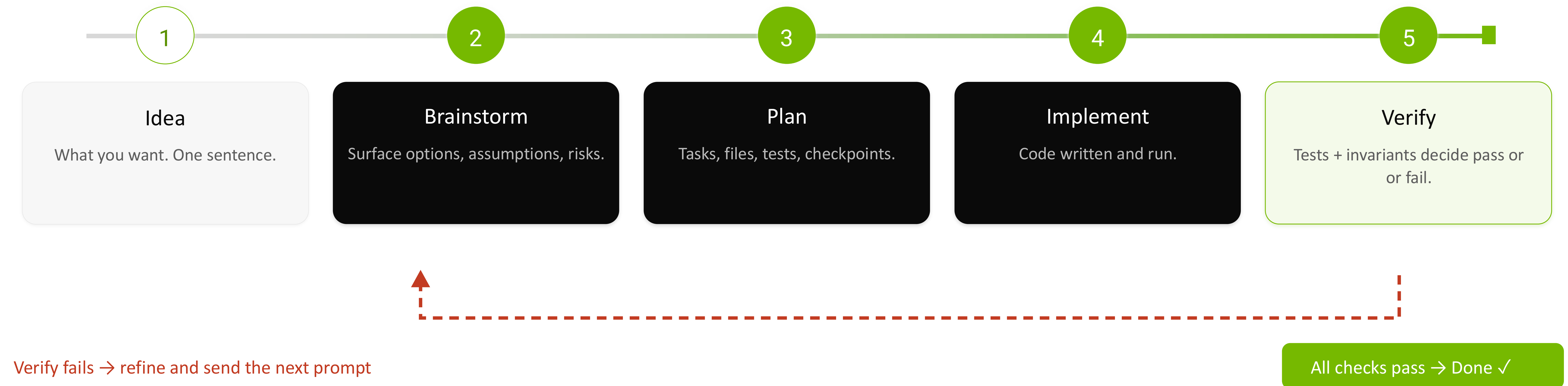
Defining Baseline Terminology

How prompt, context, agents, tools, and skills fit together



Good Agentic Coding Is a Thoughtful Process

Not a single prompt — a loop of intent, planning, building, and verifying



We'll revisit this process applied to a CUDA-Q project after a few more concepts.

Four Ways to Wire Agents Together

From a solo agent to fan-out + stitch — match the topology to the work

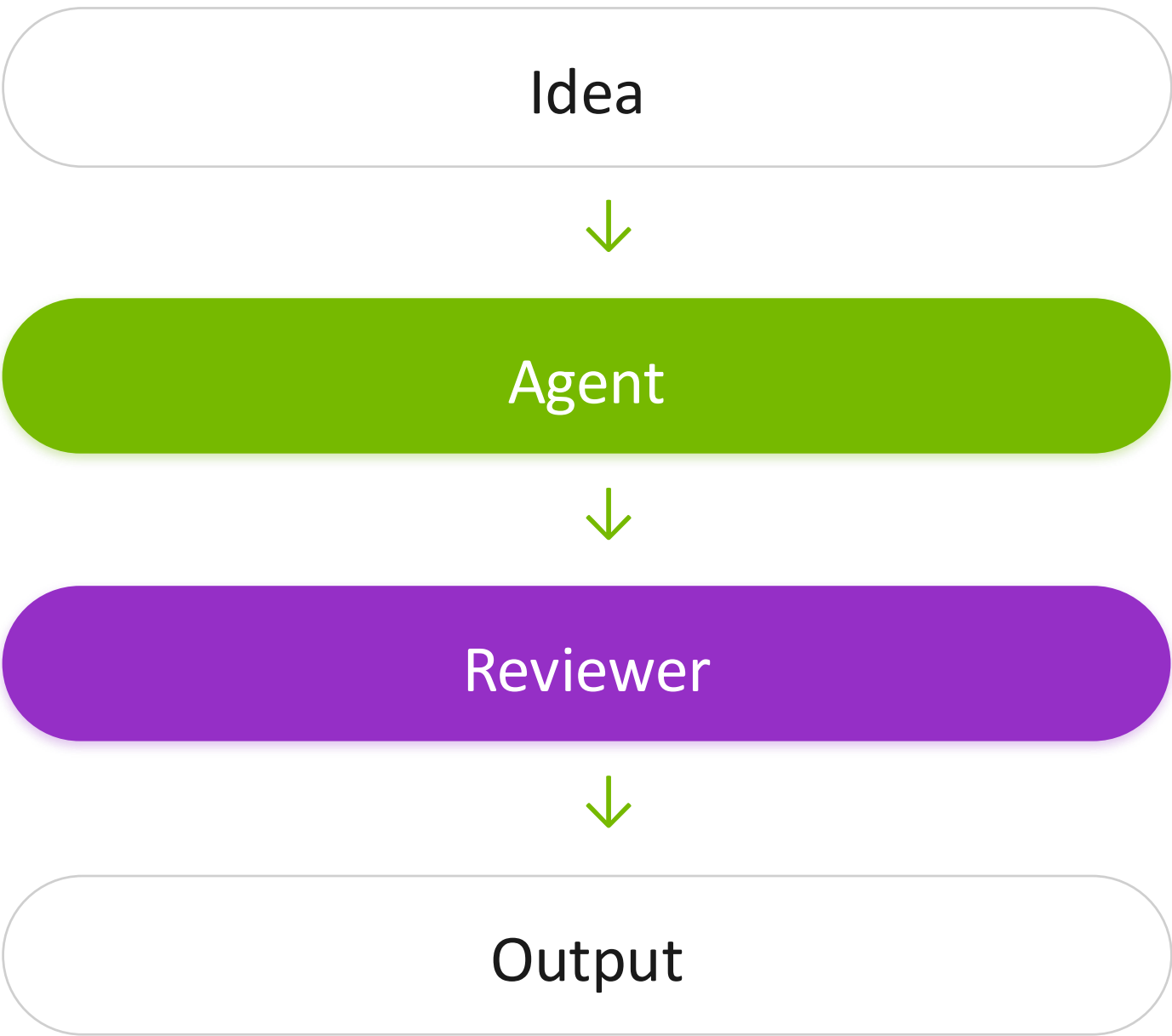
Solo agent

Simple end-to-end. Cheapest. Use when work is contained and you don't need a second opinion.



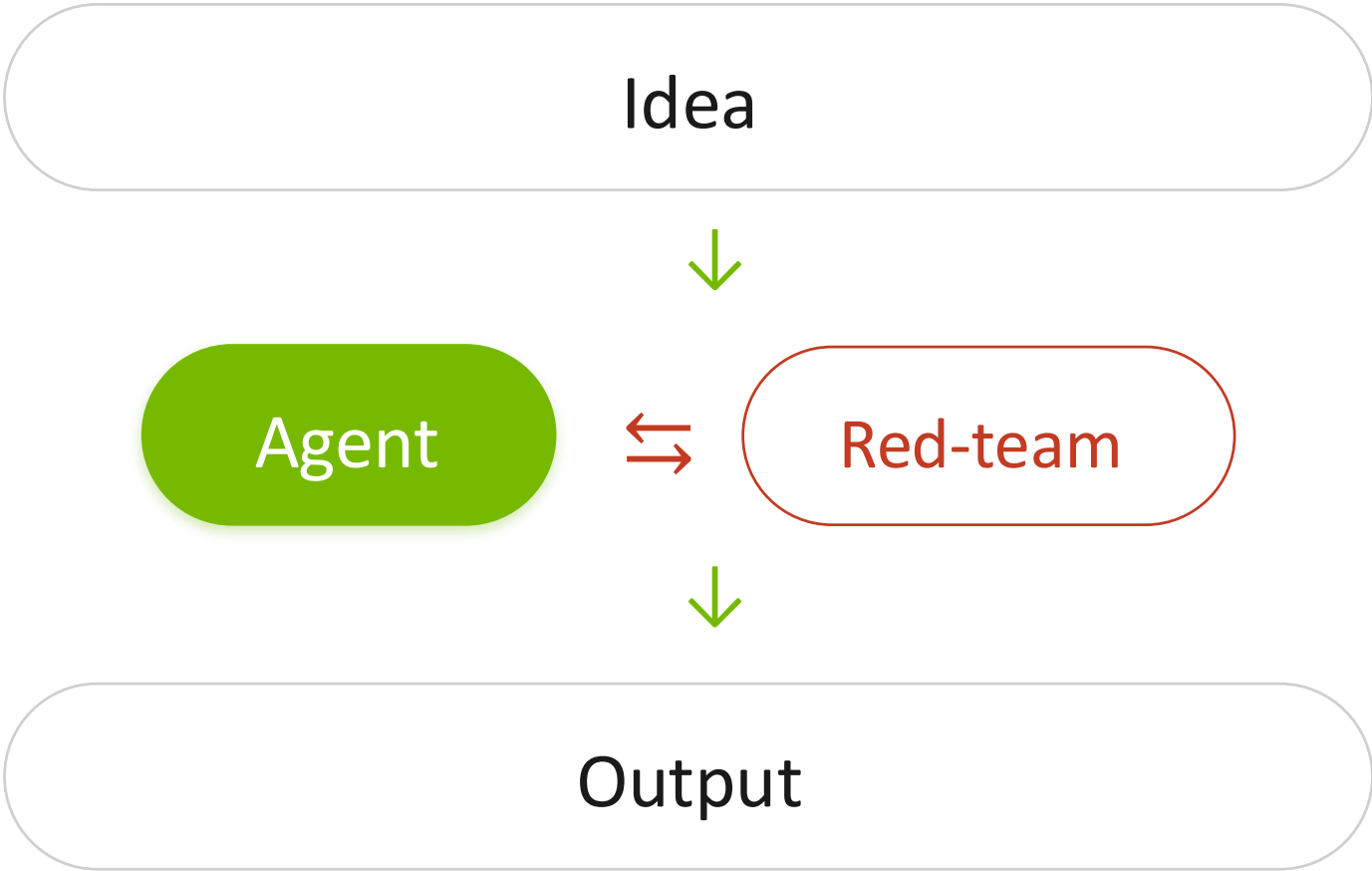
Agent + reviewer

Implementer agent, then an adversarial reviewer challenges the diff before you accept it.



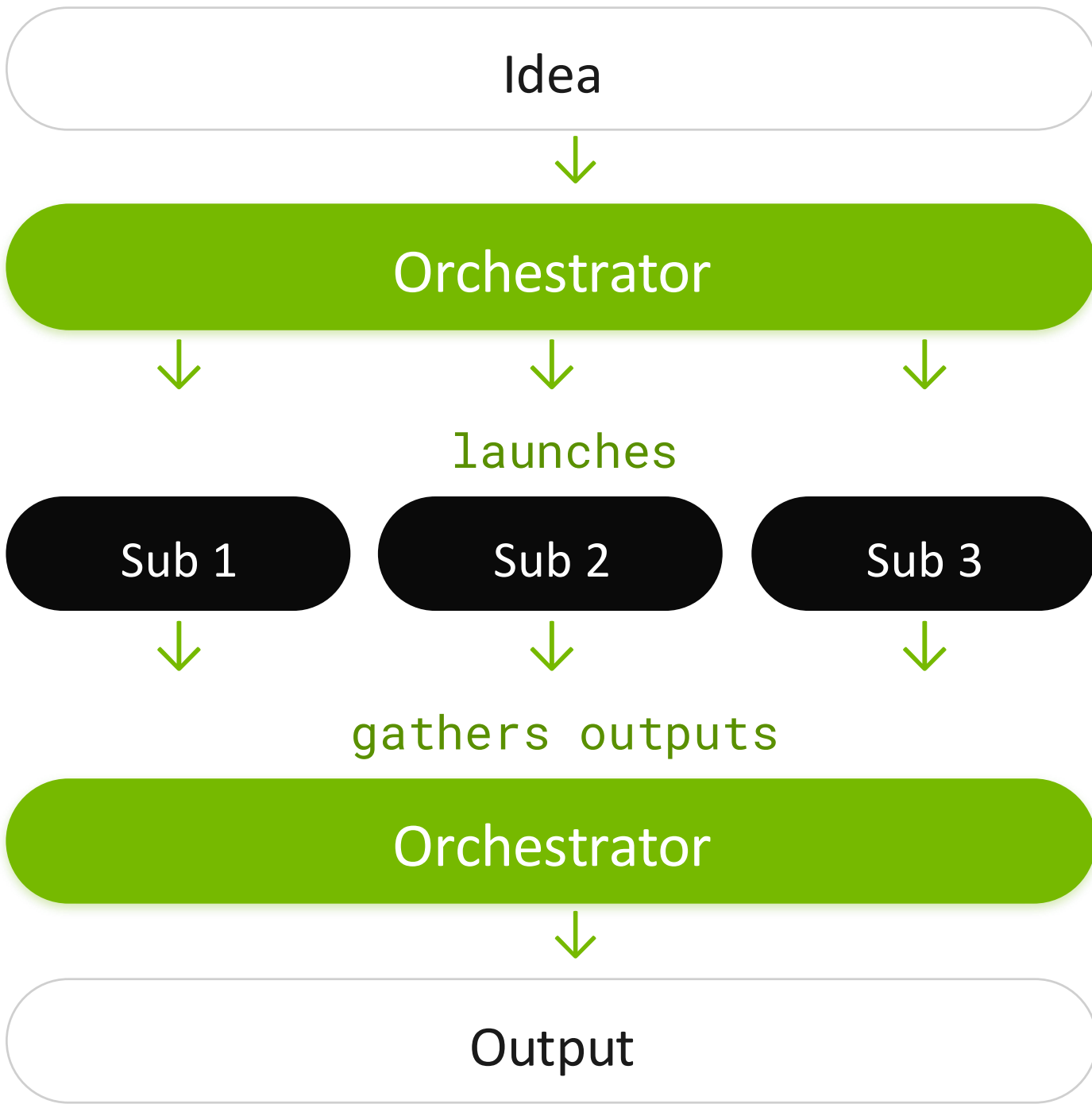
Main + red-team

Red-team agent runs alongside the implementer, probing intermediate work — not just the result.



Fan-out + stitch

An orchestrator launches sub-agents on separable tasks, gathers their outputs, then produces the result.



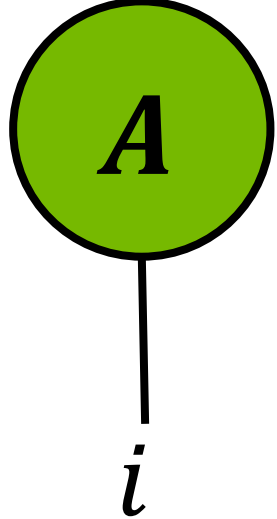
cuTensorNet

Going from quantum to tensor

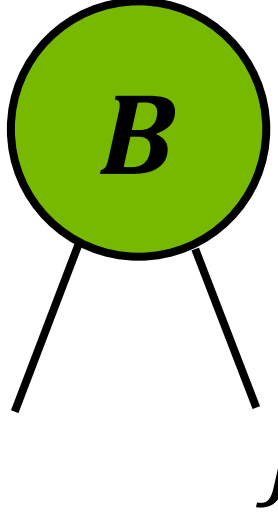
- Tensor can be interpreted as a **generalization of scalars, vectors, and matrices**

$$A = \begin{bmatrix} A_1 \\ A_2 \\ \vdots \\ A_m \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & \cdots & B_{1n} \\ \vdots & \ddots & \vdots \\ B_{m1} & \cdots & B_{mn} \end{bmatrix} \quad C = \begin{bmatrix} \begin{bmatrix} C_{111} & \cdots & C_{1n1} \\ \vdots & \ddots & \vdots \\ C_{m11} & \cdots & C_{mn1} \end{bmatrix}^1 \\ \begin{bmatrix} C_{112} & \cdots & C_{1n2} \\ \vdots & \ddots & \vdots \\ C_{m12} & \cdots & C_{mn2} \end{bmatrix}^2 \\ \begin{bmatrix} C_{113} & \cdots & C_{1n3} \\ \vdots & \ddots & \vdots \\ C_{m13} & \cdots & C_{mn3} \end{bmatrix}^3 \end{bmatrix}$$

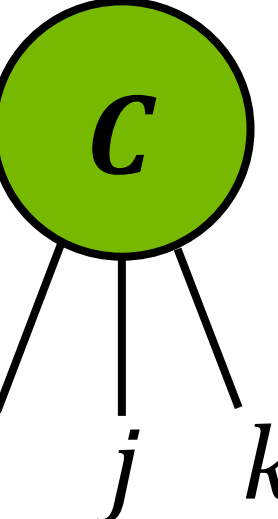
Rank-3, dimension = (m, n, 3)



1-D tensor A_i
(Vector)



2-D tensor B_{ij}
(Matrix)



3-D tensor C_{ijk}

- Tensor can **contract** together

Tensor Contraction

$$i \text{---} \textcircled{X} \text{---}^j \textcircled{Y} \text{---} k = i \text{---} \textcircled{Z} \text{---} k \quad Z_{ik} = \sum_j X_{ij} Y_{jk}$$

$$\mathbf{Z} = \begin{bmatrix} Z_{00} & Z_{01} \\ Z_{10} & Z_{11} \end{bmatrix} = \begin{bmatrix} X_{00} & X_{01} \\ X_{10} & X_{11} \end{bmatrix} \begin{bmatrix} Y_{00} & Y_{01} \\ Y_{10} & Y_{11} \end{bmatrix} = \begin{bmatrix} X_{00}Y_{00} + X_{01}Y_{10} & X_{00}Y_{01} + X_{01}Y_{11} \\ X_{10}Y_{00} + X_{11}Y_{10} & X_{10}Y_{01} + X_{11}Y_{11} \end{bmatrix}$$

- Tensor representation of a quantum circuit

$$|\Psi\rangle \text{---} \boxed{H} \text{---} = |\varphi\rangle$$

$$\textcircled{\Psi} \text{---}^j \textcircled{H} \text{---}^i = \textcircled{\varphi} \text{---}$$

$$\varphi_i = \sum_j H_{ij} \Psi_j$$

- Only the **subspaces of the tensors** need to be contracted for calculating a desired quantum state

$$|\varphi\rangle = \begin{bmatrix} \varphi_0 \\ \varphi_1 \end{bmatrix} = \begin{bmatrix} H_{00} & H_{01} \\ H_{10} & H_{11} \end{bmatrix} \begin{bmatrix} \Psi_0 \\ \Psi_1 \end{bmatrix}$$

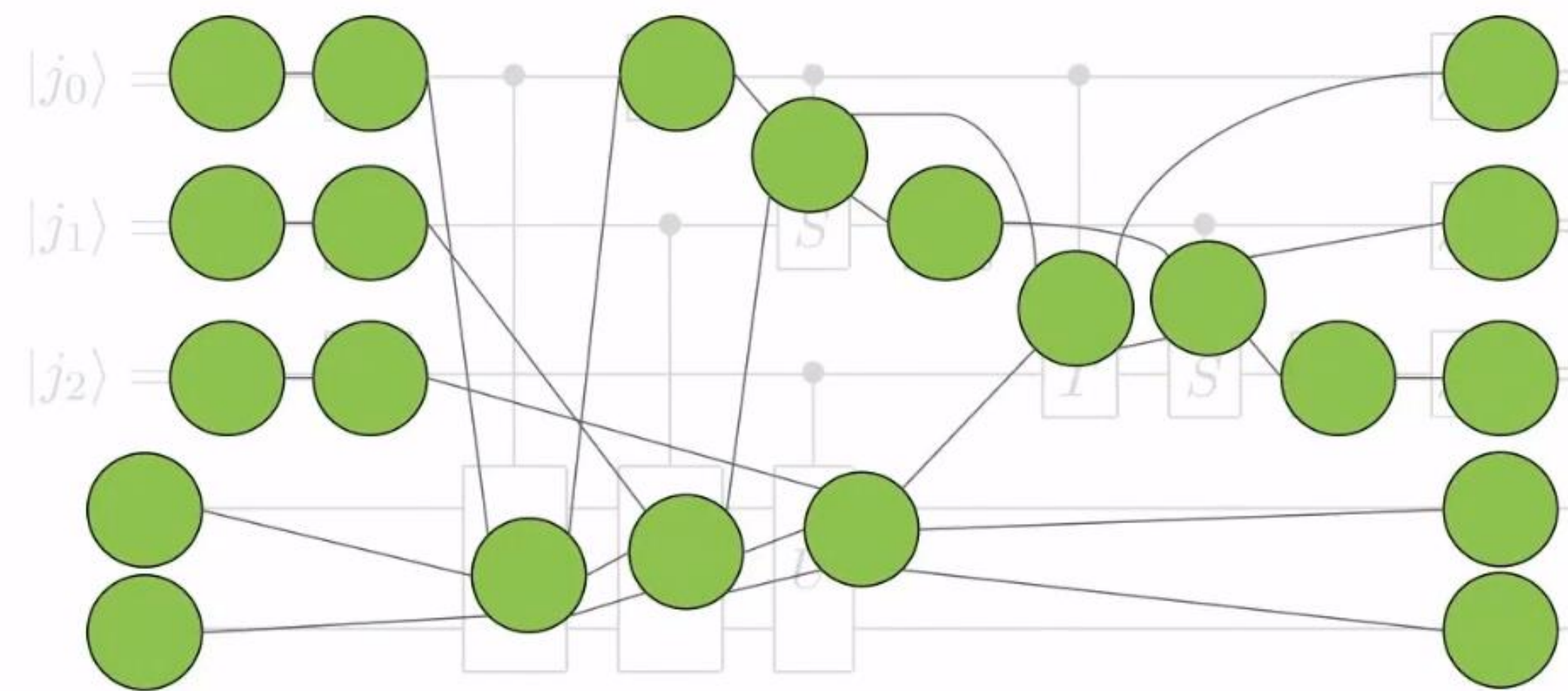
$$\varphi_1 = \sum_j \underline{H_{1j}} \Psi_j$$

Subspaces of H with $i=1$

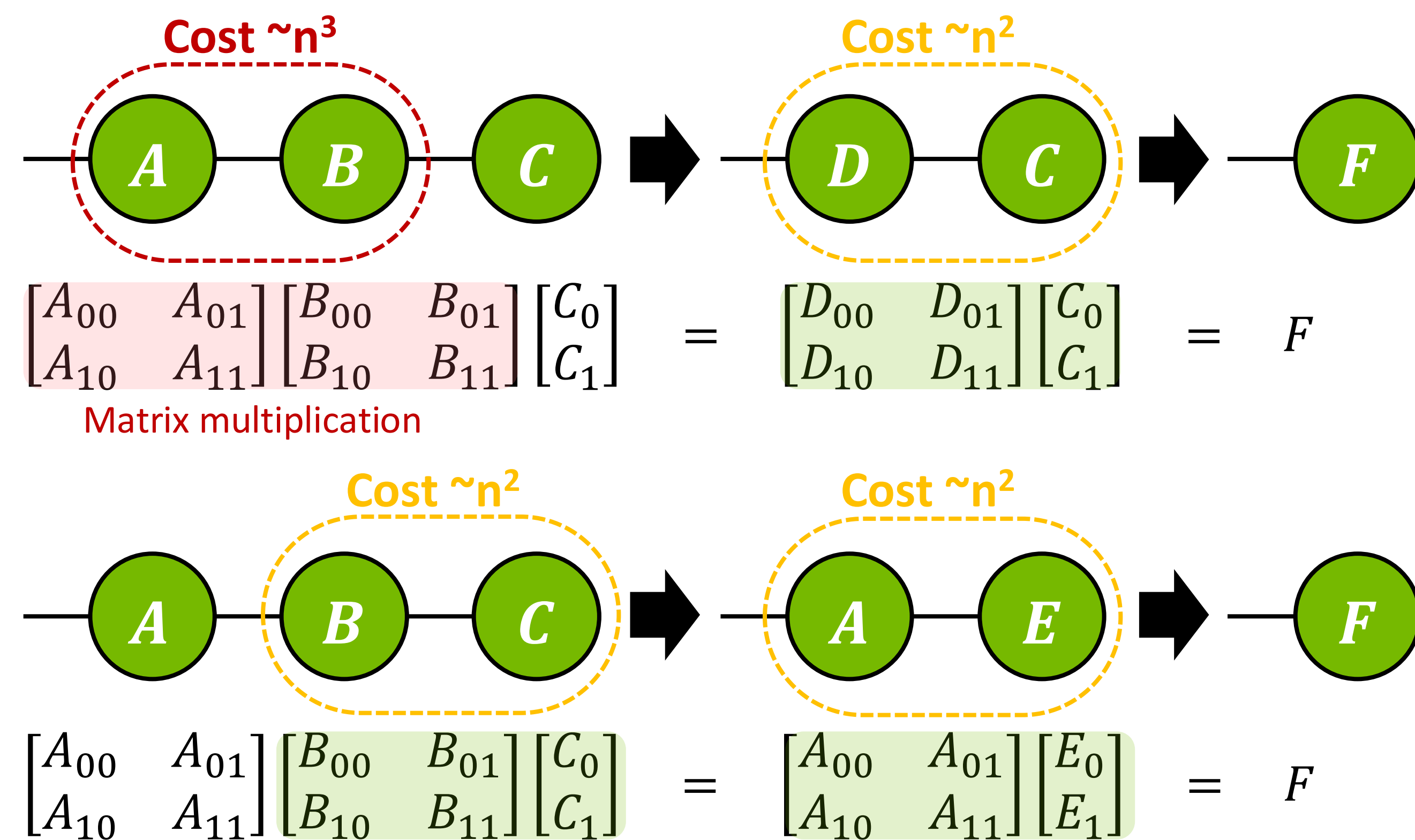
cuTensorNet

Going from tensor to optimal contraction

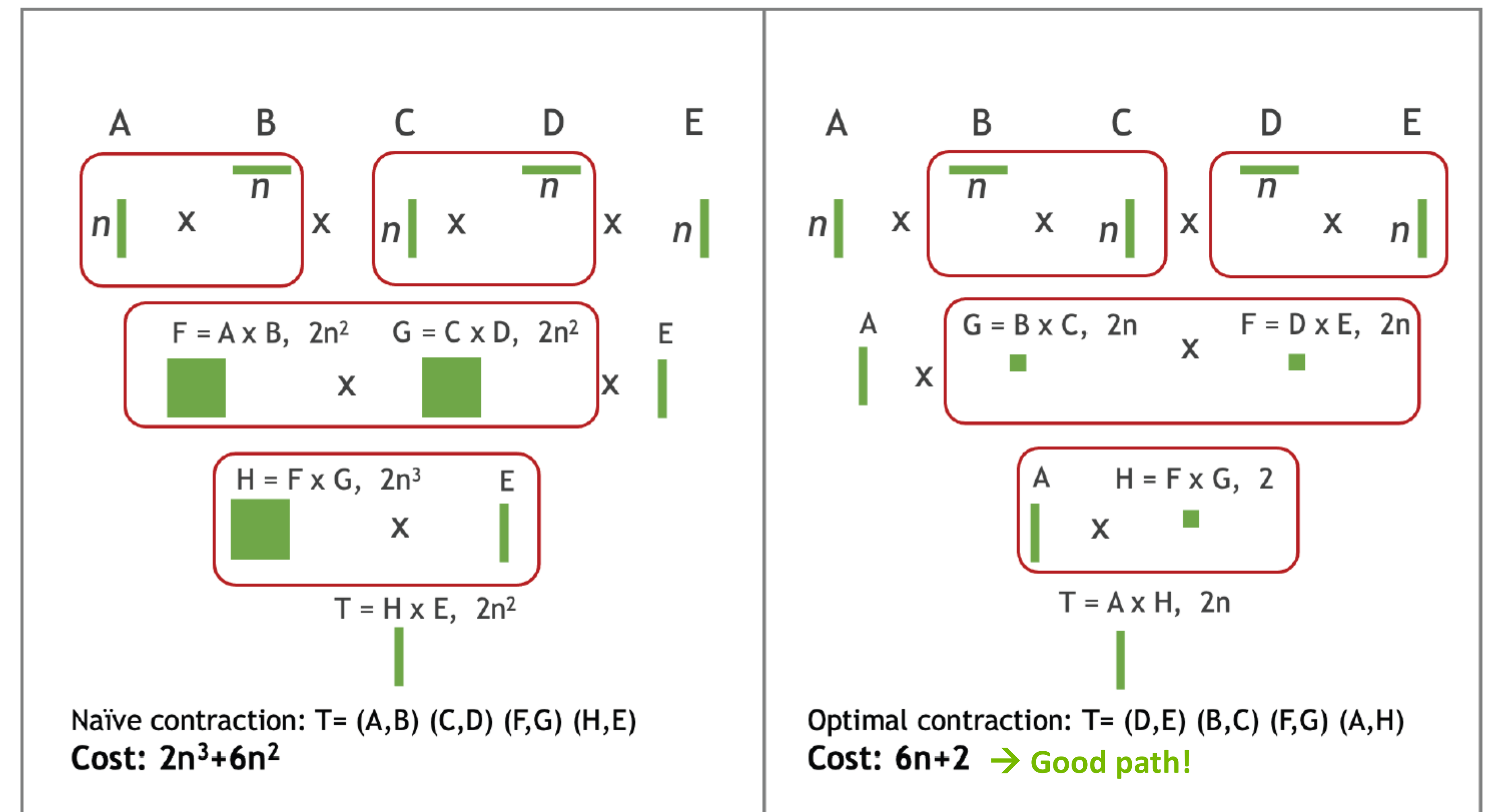
- Complex quantum circuit forms a tensor Network



- The path of the pairwise contractions does matter



- cuTensorNet gives optimal contraction path



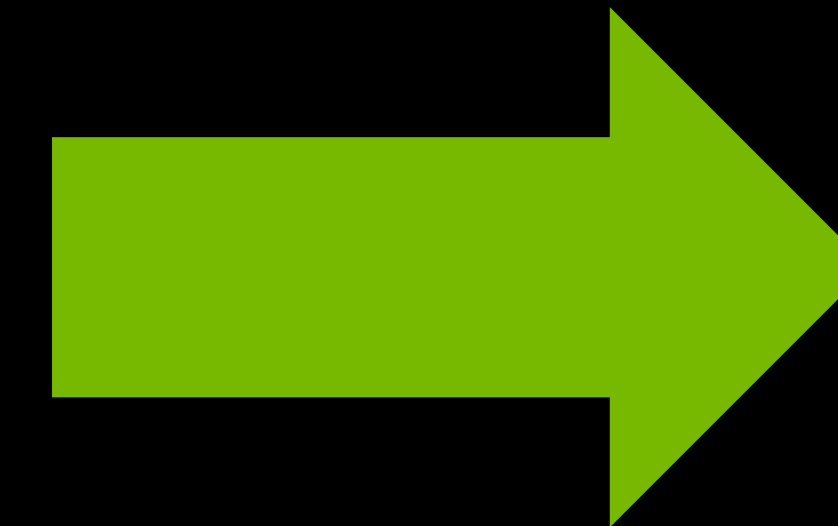
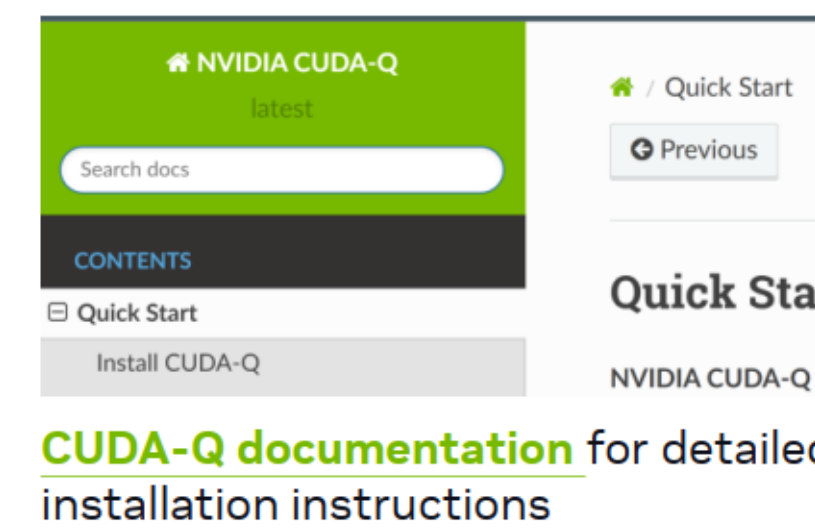
Find out More

CUDA-Q Agentic Bootcamp + Onboarding Guide

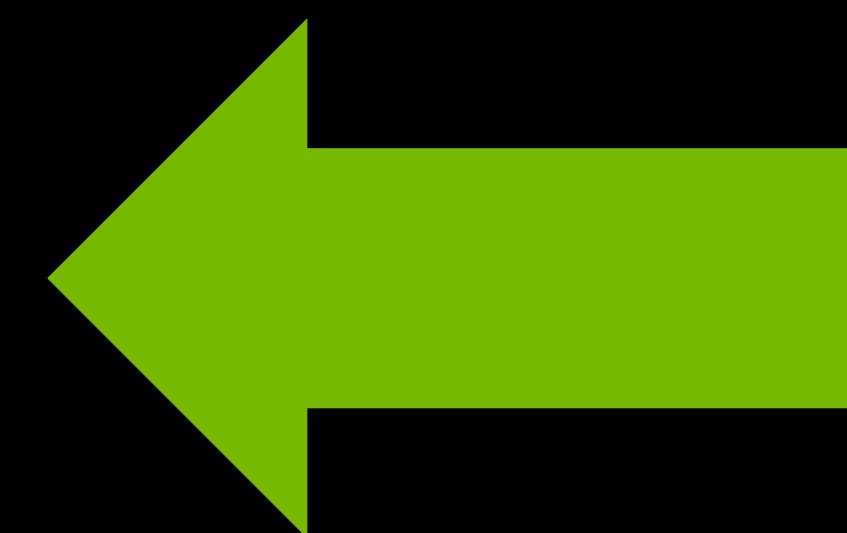
Installation Guide & Quick Start

Start your CUDA-Q journey

- **Installation by Using PyPI:**
 - Required GPU architect: *Turing, Ampere, Ada, Hopper, Blackwell*
 - Ensure that the [CUDA Toolkit](#) is installed (version >= 12.x)
(Blackwell supported for CUDA 13.x / Driver 580.65.06+ only)
 - [Driver](#) version is updated (version >= 560.35.05+)
 - Install CUDA-Q directly [via PyPI](#) (pip version >= 24.0)
- **Installation by Using Container [Recommended]:**
 - If you are using Windows, WSL (Windows Subsystem for Linux) is needed, and it should be WSL2. Follow [WSL Installation Guide](#)
 - Install Docker by visiting [Docker's website](#)
 - [Pull/Run the container image](#)
 - Container flags can be found on the [NGC page](#), for example: `nvcr.io/nvidia/quantum/cuda-quantum:cu13-0.15.0`
- **Get Started:**
 - Learn [CUDA-Q basics](#) for easy hands-on
 - [Official GitHub repository](#) for full understanding



\$10 Brev Credit
~300 NTD



A screenshot of the NVIDIA Launchables web interface. The top navigation bar includes the NVIDIA logo and links for 'GPUs', 'Agents', 'Launchables', 'Contact Us', and 'Sign In'. The main heading is 'CUDA-Q Agentic Coding Bootcamp' with a subtext 'No description available'. The interface is divided into several sections: 'Container' with a 'Docker Compose YAML' button; 'Files' with a 'GitHub Repo: cudaq-agentic-coding' link; 'Compute' showing 'Instance type' as 'NVIDIA L4 (24 GiB)' with specifications (1 GPU x 4 CPUs, 16 GiB RAM, x86_64, GCP); and 'Exposed Ports' showing 'Tunnels: jupyter-1:8888'. On the right, there's a pricing section for '\$0.75/hr' and an 'Instance type' section showing 'L4' with specifications. Below that, 'Cloud Provider' options include 'GCP' (marked as 'Creator's Choice') and 'SCALEWAY'. A 'Storage' section shows '256 GiB'. At the bottom, there's a 'Deploy Launchable' button and a consent statement: 'By clicking 'Deploy', I consent to sharing my personal and deployment data with GCP as described here.'

